

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN KHOA HỌC MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN HỌC MÁY

**Đề tài: Thuật toán Naive Bayes
và thuật toán K-means**

Giảng viên hướng dẫn: ThS. Nguyễn Đình Quý

Sinh viên thực hiện:
Lã Minh Khánh - 4004267 - 67CS1
Trịnh Quỳnh Anh - 0279367 - 67CS1
Nguyễn Hải Cường - 0174067 - 67CS1

Hà Nội, ngày 06 tháng 11 năm 2024

MỤC LỤC

1	LỜI NÓI ĐẦU	4
2	NAÏVE BAYES	5
2.1	Giới thiệu	5
2.2	Định lý Bayes	5
2.3	Thuật toán phân lớp Naïve Bayes	8
2.4	Các phân phối thường dùng cho $P(x_i c)$	9
2.4.1	Gaussian Naive Bayes	9
2.4.2	Multinomial Naive Bayes	9
2.4.3	Bernoulli Naive Bayes	10
2.5	Ví dụ	10
2.6	Ưu điểm và nhược điểm	12
2.7	Bài tập thực nghiệm	13
2.7.1	Giới thiệu	13
2.7.2	Tập được cung cấp	13
2.7.3	Mô tả dữ liệu	13
2.7.4	Hướng giải quyết	14
2.7.5	Huấn luyện	15
2.7.6	Phân lớp	16
2.7.7	Yêu cầu	17
2.7.8	Lời giải	19
2.8	Tài liệu tham khảo	25
3	K-MEANS	26
3.1	Giới thiệu	26
3.2	Thuật toán K-means	27
3.2.1	Thuật toán này hoạt động như nào?	27
3.2.2	Phương pháp Elbow	31
3.3	Phân loại K-means	32
3.3.1	K-means	32
3.3.2	K-means++	32
3.3.3	K-means lô nhỏ	32
3.3.4	K-medoids	33
3.4	Ví dụ	34
3.5	Ưu điểm và nhược điểm	40

3.6	Bài tập thực nghiệm	41
3.6.1	Giới thiệu	41
3.6.2	Tập được cung cấp	41
3.6.3	Hướng giải quyết	41
3.6.4	Yêu cầu	43
3.6.5	Lời giải	43
3.7	Tài liệu tham khảo	60
4	LỜI KẾT	61

LỜI NÓI ĐẦU

Chúng em xin chân thành cảm ơn thầy Nguyễn Đình Quý đã trang bị những kiến thức quý báu cho chúng em trong suốt quá trình học tập môn học phần Học máy - Machine Learning. Chính nhờ công giảng dạy, chỉ bảo tận tình của thầy mà chúng em mới có những kiến thức cơ bản để bước vào chuyên ngành và đặt những bước chân tiếp theo trên con đường học tập, vận dụng và sáng tạo ra những sản phẩm hữu ích góp phần phục vụ các lĩnh vực khác nhau của đời sống.

Chúng em xin chân thành cảm ơn thầy đã tận tình hướng dẫn, chỉ bảo cho chúng em trong suốt quá trình thực hiện đề tài này.

Chúng em xin gửi lời cảm ơn đến gia đình, bạn bè, các anh chị tiền bối đã động viên và cung cấp các tài liệu hữu ích cho nhóm để chúng em hoàn thành đề tài một cách chính chu nhất.

Mặc dù đã cố gắng nỗ lực thực hiện đề tài với quyết tâm cao nhưng bài làm chắc chắn không thể tránh khỏi thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp của các thầy để bài tập và kiến thức của chúng em ngày càng hoàn thiện hơn và rút kinh nghiệm trong những lần làm đề tài sau ạ.

Chúng em xin chân thành cảm ơn!

Ngày 6 tháng 11 năm 2024

Nhóm 5

LÃ MINH KHÁNH

NGUYỄN HẢI CƯỜNG

TRỊNH QUỲNH ANH

NAÏVE BAYES

Chúng ta đều quá quen với ứng dụng Gmail của Google rồi. Mỗi lần chúng ta đều đăng kí tham gia câu lạc bộ nào đấy ở trường hay dự án cộng đồng bên ngoài, khi mà nhận kết quả vòng đơn hay vòng phỏng vấn, bên câu lạc bộ sẽ gửi email thông báo kết quả đến với chúng ta, và bình thường sẽ luôn kèm cả câu: "Tuyệt đối nhớ check cả thư mục rác để không bị bỏ sót thư báo kết quả nhé!". Có bao giờ chúng ta tò mò rằng khi mà có ai đó gửi thư cho bọn mình, hệ thống của Google sẽ nhận diện trong bức thư đấy có dấu hiệu nào để ném thẳng nó vào khu vực mục thư rác hay không?

Đó là một trong những ứng dụng phổ biến rộng rãi của thuật toán phân lớp nổi tiếng mà nhóm chúng em sẽ giới thiệu ở bài tập lớn môn Machine Learning, Naïve Bayes. Naïve Bayes dựa trên nguyên tắc của Định lý Bayes, một công cụ quan trọng trong lý thuyết xác suất. Nó giúp chúng ta dự đoán một dữ liệu thuộc vào nhóm nào dựa trên các đặc trưng của nó. Ưu điểm của thuật toán này là đơn giản và hiệu quả, đặc biệt khi xử lý lượng dữ liệu lớn.

Chúng ta cùng đi sâu về lý thuyết thuật toán Naïve Bayes nhé!

2.1 Giới thiệu

Thuật toán Naïve Bayes là một thuật toán học máy đơn giản nhưng hiệu quả, được sử dụng để gán nhãn cho dữ liệu dựa trên các đặc trưng của nó. Nó hoạt động bằng cách tính toán xác suất để một đối tượng thuộc về một lớp nhất định, dựa trên giả định rằng các đặc trưng là độc lập với nhau.

Thuật toán Naïve Bayes sử dụng định lý Bayes để phân loại dữ liệu. Nó giúp chúng ta dự đoán khả năng một dữ liệu thuộc vào một lớp nhất định dựa trên các đặc trưng của nó.

Naïve Bayes - nghĩa đen của Naïve còn có nghĩa là "ngây thơ", vì nó giả định một cách đơn giản rằng các đặc trưng của dữ liệu không có mối quan hệ với nhau. Mặc dù đây là giả định táo bạo và không thực tế đối với tập dữ liệu thực tế, nhưng thuật toán này vẫn được sử dụng rộng rãi nhờ vào hiệu quả tính toán và khả năng xử lý dữ liệu lớn.

2.2 Định lý Bayes

Trước mắt chúng ta cần hiểu, Định lý Bayes là gì?

Định lý Bayes cho phép tính xác suất xảy ra của một sự kiện ngẫu nhiên A khi biết sự kiện liên quan B đã xảy ra. Chúng ta có bao giờ tự hỏi là khi trời nhiều mây,

khả năng trời mưa lại cao hơn? Những câu hỏi như này thường rất hay gặp ở những dạng bài toán liên quan đến định lý Bayes, đặc biệt là môn học liên quan đến Thống kê. Khi mà ta đã biết trời đang nhiều mây (sự kiện B), từ đó có khả năng sẽ xảy ra sự kiện A - là trời mưa, và mình cần tính toán xem khả năng đây sẽ xảy ra là bao nhiêu? Từ đó ta có ký hiệu là $\mathbb{P}(A|B)$, hay còn gọi là "xác suất của A nếu có B ". Từ đó đại lượng này ta được gọi là **xác suất có điều kiện** hay **xác suất hậu nghiệm** vì nó được rút ra từ giá trị được cho của B hoặc phụ thuộc vào giá trị đó.

Theo định lý Bayes, xác suất xảy ra sự kiện A khi sự kiện B đã diễn ra sẽ còn tùy thuộc vào 3 yếu tố như sau

- Xác suất biên duyên - hay còn được biến tên gọi là **Marginal distribution** hay còn có tên gọi khác là Xác suất tiên nghiệm. Khi ta đã nói "tiên nghiệm" có nghĩa là chỉ có xác suất A xảy ra thôi và không có liên quan gì đến thông tin nào đến sự kiện B đã xảy ra cả. Ta sẽ gọi ký hiệu này là $\mathbb{P}(A)$ hay tên gọi là "xác suất của A ".
- Hằng số chuẩn hóa - tên tiếng Anh là **Normalizing constant**, ám chỉ rằng chỉ có xác suất B xảy ra và hoàn toàn độc lập trước sự kiện A đã xảy ra, chính vì vậy đại lượng này luôn giống nhau. Chúng ta sẽ gọi là "xác suất của B " hay ký hiệu là $\mathbb{P}(B)$.
- **Likelihood**, hay được biết đến ký hiệu $\mathbb{P}(B|A)$, được đọc là "xác suất của B nếu có A ", đại lượng này chỉ ra rằng sự kiện B sẽ xảy ra khi biết sự kiện A đã xảy ra. Lưu ý: khả năng xảy ra sự kiện A khi sự kiện B đã diễn ra và khả năng xảy ra sự kiện B khi sự kiện A đã diễn ra hoàn toàn độc lập với nhau, nhưng chúng ta rất dễ nhầm lẫn, nên chúng ta sẽ cần phải chú ý những điều này.

Từ 3 yếu tố đại lượng được nêu trên, ta có công thức tính xác suất của A khi biết trước sự kiện B đã diễn ra như sau (công thức 2.1):

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(B|A)\mathbb{P}(A)}{\mathbb{P}(B)} = \frac{\text{likelihood} \times \text{marginal_distribution}}{\text{normalizing_constant}} \quad (2.1)$$

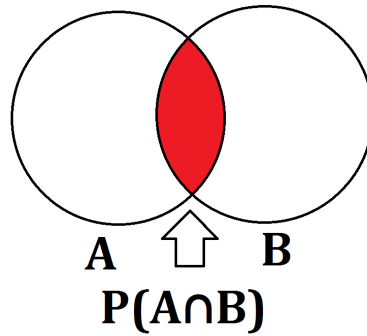
Như vậy, định lý Bayes là một cách để tìm một xác suất khi ta đã có dữ kiện về một xác suất khác có liên quan.

Ta sẽ chứng minh công thức như sau:

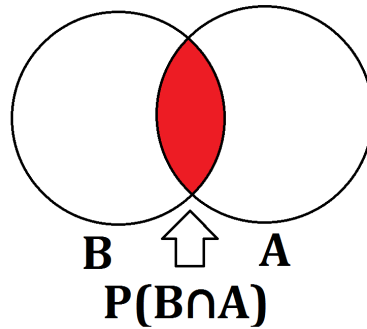
$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}, \mathbb{P}(B|A) = \frac{\mathbb{P}(B \cap A)}{\mathbb{P}(A)} \quad (2.2)$$

Từ hai công thức 2.2 này, ta vẽ 2 đường tròn A với B giao điểm với nhau, tiếp

đó ta cũng vẽ hai đường tròn B và A giao điểm với nhau, những ảnh này được minh họa qua hai ảnh 2.1 và 2.2 như sau:



Hình 2.1: Hình ảnh minh họa A giao với B



Hình 2.2: Hình ảnh minh họa B giao với A

Từ đây ta có thể suy ra:

$$\mathbb{P}(A \cap B) = \mathbb{P}(B \cap A) \quad (2.3)$$

Chính vì thế, ta thay $P(B \cap A)$ ở công thức $\mathbb{P}(B|A)$ thành $\mathbb{P}(A \cap B)$, từ đó ta có:

$$\mathbb{P}(B|A) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(A)} \quad (2.4)$$

Nhân cả 2 vế công thức $\mathbb{P}(B|A)$ với $\mathbb{P}(A)$, ở vế bên phải, ta rút gọn $\mathbb{P}(A)$ đi, trở thành công thức:

$$\mathbb{P}(B|A)\mathbb{P}(A) = \mathbb{P}(A \cap B) \quad (2.5)$$

Thế là từ công thức $\mathbb{P}(A|B)$, ta thay $\mathbb{P}(A \cap B)$ như sau:

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(B|A)\mathbb{P}(A)}{\mathbb{P}(B)} \text{ (điều phải chứng minh)} \quad (2.6)$$

2.3 Thuật toán phân lớp Naïve Bayes

Xét bài toán classification với C classes $1, 2, \dots, C$. Giả sử có một điểm dữ liệu $X \in \mathbb{R}^d$. Hãy tính xác suất để điểm dữ liệu này rơi vào class $c \in C$. Nói cách khác, hãy tính:

$$\mathbb{P}(c|X) \quad (2.7)$$

Tức tính xác suất để đầu ra là class c biết rằng đầu vào là vector X . Với X được cho là:

$$X = (X_1, X_2, X_3, \dots, X_n) \quad (2.8)$$

Từ (2.7) ta áp dụng định lý Bayes vào, ta có:

$$\mathbb{P}(c|X) = \frac{\mathbb{P}(X|c)\mathbb{P}(c)}{\mathbb{P}(X)} \quad (2.9)$$

Kết hợp (2.8) với (2.9) vào ta cũng có:

$$\mathbb{P}(c|X_1, X_2, X_3, \dots, X_n) = \frac{\mathbb{P}(X_1|c) \times \mathbb{P}(X_2|c) \times \mathbb{P}(X_3|c) \times \dots \times \mathbb{P}(X_n|c) \times \mathbb{P}(c)}{\mathbb{P}(X_1) \times \mathbb{P}(X_2) \times \mathbb{P}(X_3) \times \dots \times \mathbb{P}(X_n)} \quad (2.10)$$

Đối với tất cả các mục trong tập dữ liệu, mẫu số không thay đổi, giữ nguyên và không phụ thuộc vào c . Do đó, mẫu số có thể được loại bỏ. Từ đó ta có phương trình như sau:

$$\begin{aligned} \mathbb{P}(c|X_1, X_2, X_3, \dots, X_n) &\propto \mathbb{P}(X_1|c) \times \mathbb{P}(X_2|c) \times \mathbb{P}(X_3|c) \times \dots \times \mathbb{P}(X_n|c) \\ \Rightarrow \mathbb{P}(c|X_1, X_2, X_3, \dots, X_n) &\propto \mathbb{P}(c) \times \prod_{i=1}^n \mathbb{P}(x_i|c) \end{aligned} \quad (2.11)$$

Bây giờ chúng ta cần tạo một mô hình phân loại. Đối với điều này, chúng ta cần tìm xác suất của tập hợp đầu vào nhất định cho tất cả các giá trị có thể có của biến lớp c và chọn đầu ra với xác suất tối đa. Điều này có thể được biểu diễn bằng toán học như sau:

$$y = \arg \max_c \mathbb{P}(c) \prod_{i=1}^n \mathbb{P}(x_i|c), \quad c \in \{1, \dots, C\} \quad (2.12)$$

Khi n lớn và các xác suất nhỏ, biểu thức (2.12) ở vế bên phải sẽ là một số rất nhỏ, khi tính toán có thể gặp sai số. Để giải quyết việc này, biểu thức (2.12) được viết lại dưới dạng tương đương bằng cách lấy $\log$ của vế phải:

$$y = \arg \max_c \mathbb{P}(c) \prod_{i=1}^n \mathbb{P}(x_i|c), \quad c \in \{1, \dots, C\} \quad (2.13)$$

$$\Leftrightarrow y = \log(\mathbb{P}(c)) + \sum_{i=1}^n \mathbb{P}(x_i|c)$$

2.4 Các phân phối thường dùng cho $\mathbf{P}(\mathbf{x}_i|c)$

Việc tính toán $\mathbb{P}(x_i|c)$ phụ thuộc vào loại dữ liệu. Có ba loại được sử dụng phổ biến là: Gaussian Naive Bayes, Multinomial Naive Bayes, và Bernoulli Naive.

2.4.1 Gaussian Naive Bayes

Mô hình này được sử dụng chủ yếu trong loại dữ liệu mà các thành phần là các biến liên tục. Với mỗi chiều dữ liệu i và một class c , x_i tuân theo một phân phối chuẩn có kỳ vọng μ_{ci} và phương sai σ_{ci}^2 :

$$p(x_i|c) = p(x_i|\mu_{ci}, \sigma_{ci}^2) = \frac{1}{\sqrt{2\pi\sigma_{ci}^2}} \exp\left(-\frac{(x_i - \mu_{ci})^2}{2\sigma_{ci}^2}\right) \quad (2.14)$$

Trong đó, bộ tham số $\theta = \{\mu_{ci}, \sigma_{ci}^2\}$ được xác định bằng Maximum Likelihood:

$$(\mu_{ci}, \sigma_{ci}^2) = \arg \max_{\mu_{ci}, \sigma_{ci}^2} \prod_{n=1}^N p(x_i^{(n)}|\mu_{ci}, \sigma_{ci}^2) \quad (2.15)$$

2.4.2 Multinomial Naive Bayes

Mô hình này chủ yếu được sử dụng trong phân loại văn bản mà feature vectors được tính bằng Bags of Words. Lúc này, mỗi văn bản được biểu diễn bởi một vector có độ dài d chính là số từ trong từ điển. Giá trị của thành phần thứ i trong mỗi vector chính là số lần từ thứ i xuất hiện trong văn bản đó.

Khi đó, $\mathbb{P}(x_i|c)$ tỉ lệ với tần suất từ thứ i (hay feature thứ i cho trường hợp tổng quát) xuất hiện trong các văn bản của class c . Giá trị này có thể được tính bằng cách:

$$\lambda_{ci} = p(x_i|c) = \frac{N_{ci}}{N_c} \quad (2.16)$$

Trong đó:

- N_{ci} là tổng số lần từ thứ i xuất hiện trong các văn bản của class c nó được tính là tổng của tất cả các thành phần thứ i của các feature vectors ứng với class c .
- N_c là tổng số từ (kể cả lặp) xuất hiện trong class c . Nói cách khác, nó bằng

tổng độ dài của toàn bộ các văn bản thuộc vào class c . Có thể suy ra rằng $N_c = \sum_{i=1}^d N_{ci}$, từ đó $\sum_{i=1}^d \lambda_{ci} = 1$.

Cách tính này có một hạn chế là nếu có một từ mới chưa bao giờ xuất hiện trong class c thì biểu thức (2.16) sẽ bằng 0, điều này dẫn đến vế phải của biểu thức (2.12) bằng 0 bất kể các giá trị còn lại có lớn thế nào. Việc này sẽ dẫn đến kết quả không chính xác.

Để giải quyết việc này, một kỹ thuật được gọi là *Laplace smoothing* được áp dụng:

$$\hat{\lambda}_{ci} = \frac{N_{ci} + \alpha}{N_c + d\alpha} \quad (2.17)$$

Với α là một số dương, thường bằng 1, để tránh trường hợp tử số bằng 0. Mẫu số được cộng với $d\alpha$ để đảm bảo tổng xác suất $\sum_{i=1}^d \hat{\lambda}_{ci} = 1$.

Như vậy, mỗi class c sẽ được mô tả bởi bộ các số dương có tổng bằng 1: $\hat{\lambda}_c = \{\hat{\lambda}_{c1}, \dots, \hat{\lambda}_{cd}\}$.

2.4.3 Bernoulli Naive Bayes

Mô hình này được áp dụng cho các loại dữ liệu mà mỗi thành phần là một giá trị binary - bằng 0 hoặc 1. Ví dụ: cũng với loại văn bản nhưng thay vì đếm tổng số lần xuất hiện của 1 từ trong văn bản, ta chỉ cần quan tâm từ đó có xuất hiện hay không.

Khi đó, $\mathbb{P}(x_i|c)$ được tính bằng:

$$p(x_i|c) = p(i|c)^{x_i} (1 - p(i|c))^{1-x_i} \quad (2.18)$$

với $\mathbb{P}(i|c)$ có thể được hiểu là xác suất từ thứ i xuất hiện trong các văn bản của class c .

2.5 Ví dụ

Chúng ta có cơ sở dữ liệu như sau:

ID	Tuổi	Thu nhập	Sinh viên	Đánh giá tín dụng	Mua máy tính
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle	low	yes	excellent	yes
8	youth	medium	no	fair	yes
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle	medium	no	excellent	yes
13	middle	high	yes	fair	yes
14	senior	medium	no	excellent	no

Giả sử ta có một khách hàng mới X có các thuộc tính như sau:

$$X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit_rating} = \text{fair})$$

Bây giờ cần xác định xem khách hàng X có thuộc lớp C_{yes} (mua máy tính) hay không, trước hết ta sẽ tính xác suất người dùng có chọn mua máy tính hay không?

$$\mathbb{P}(C_{yes}) = \frac{9}{14} \approx 0.643 \quad \mathbb{P}(C_{no}) = \frac{5}{14} \approx 0.357$$

Ta sẽ tính các xác suất thành phần có lựa chọn mua máy tính không, như sau:

$$P(\text{age} = \text{youth} | C_{yes}) = \frac{2}{9} \approx 0.222$$

$$P(\text{age} = \text{youth} | C_{no}) = \frac{3}{5} = 0.6$$

$$P(\text{income} = \text{medium} | C_{yes}) = \frac{4}{9} \approx 0.444$$

$$P(\text{income} = \text{medium} | C_{no}) = \frac{2}{5} = 0.4$$

$$P(\text{student} = \text{yes} | C_{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{student} = \text{yes} | C_{no}) = \frac{1}{5} = 0.2$$

$$P(\text{credit_rating} = \text{fair} | C_{yes}) = \frac{6}{9} \approx 0.667$$

$$P(\text{credit_rating} = \text{fair} | C_{no}) = \frac{2}{5} = 0.4$$

Tiếp theo, ta sẽ tính $\mathbb{P}(X | C_i)$:

$$P(X_i | C_{yes}) = 0.222 \times 0.444 \times 0.667 \times 0.667 \approx 0.044$$

$$P(X_i | C_{no}) = 0.6 \times 0.4 \times 0.2 \times 0.4 = 0.0192$$

Cuối cùng, ta đã có đủ dữ kiện để tính $\mathbb{P}(C_i | X)$:

$$\mathbb{P}(C_{yes}|X) = 0.643 \times 0.044 = 0.28292$$

$$\mathbb{P}(C_{no}|X) = 0.357 \times 0.0192 \approx 0.07$$

Từ kết quả này ta thấy $\mathbb{P}(C_{yes}|X)$ có giá trị lớn nhất, do đó thuật toán Bayes sẽ kết luận là khách hàng X sẽ mua máy tính.

2.6 Ưu điểm và nhược điểm

Thuật toán Naïve Bayes	Ưu điểm	Nhược điểm
	• Nó đơn giản và dễ dàng để triển khai. • Nó không yêu cầu nhiều dữ liệu huấn luyện. • Nó xử lý cả dữ liệu liên tục và rời rạc. • Nó có khả năng mở rộng cao với số lượng các biến dự đoán và điểm dữ liệu. • Nó nhanh và có thể được sử dụng để đưa ra dự đoán thời gian thực. • Nó không nhạy cảm với các đặc trưng không liên quan. • Nó hội tụ nhanh hơn các mô hình khác, đặc biệt là các mô hình phân biệt.	• Vấn đề tần số bằng 0: Khi một giá trị nào đó của một thuộc tính danh mục không xuất hiện trong tập dữ liệu huấn luyện, thuật toán Naive Bayes sẽ gán cho nó xác suất bằng 0. Điều này có thể dẫn đến các vấn đề trong quá trình dự đoán, đặc biệt khi gặp phải dữ liệu mới chưa từng thấy. Phương án giải quyết vấn đề đã được nêu rõ ở biểu thức (2.17). • Thuật toán này giả định rằng tất cả các thuộc tính là độc lập, điều này hiếm khi xảy ra trong thực tế. Điều này sẽ hạn chế việc áp dụng thuật toán này trong các tình huống thực tế. • Đôi khi thuật toán này sẽ ước tính sai, vì vậy bạn không nên quá tin vào các kết quả xác suất mà nó đưa ra.

2.7 Bài tập thực nghiệm

2.7.1 Giới thiệu

Nhận dạng chữ viết tay là khả năng của máy tính để hiểu và diễn giải văn bản viết tay thành các ký tự. Trong bài tập này, chúng ta sẽ thử nhận dạng các con số từ hình ảnh. Để thực hiện nhiệm vụ này, chúng ta sẽ sử dụng phân loại Naive Bayes.

Phân loại Naive Bayes là một họ các phân loại xác suất dựa trên định lý Bayes. Các thuật toán này hoạt động bằng cách kết hợp các xác suất mà một ví dụ thuộc về một lớp dựa trên giá trị của một tập hợp các đặc trưng. Trong trường hợp này, chúng ta sẽ kiểm tra xem hình ảnh có thuộc lớp các chữ số từ 0 đến 9 hay không dựa trên trạng thái của các pixel trong hình ảnh.

2.7.2 Tập được cung cấp

Chúng tôi cung cấp các tệp dữ liệu đại diện cho các hình ảnh chữ viết tay cùng với phân loại chính xác của các hình ảnh này tên "digitdata.zip".

Tệp zip này chứa các mục sau:

- readme.txt: Mô tả các tệp trong tệp zip.
- testimages: 1000 hình ảnh để kiểm tra.
- testlabels: Lớp chính xác cho mỗi hình ảnh kiểm tra.
- trainingimages: 5000 hình ảnh để huấn luyện.
- traininglabels: Lớp chính xác cho mỗi hình ảnh huấn luyện.

Chúng ta sẽ sử dụng các hình ảnh huấn luyện để huấn luyện bộ phân loại của chúng ta, cụ thể là để tính toán các xác suất mà chúng ta sẽ sử dụng và sau đó cố gắng nhận dạng các hình ảnh kiểm tra.

2.7.3 Mô tả dữ liệu

Dữ liệu hình ảnh cho tập dữ liệu này bao gồm các hình ảnh có kích thước 28x28 pixel được lưu dưới dạng văn bản. Mỗi hình ảnh có 28 dòng văn bản, mỗi dòng chứa 28 ký tự. Giá trị của các pixel được mã hóa bằng ' ' cho màu trắng, '+' cho màu xám và '' cho màu đen. Bạn có thể xem các hình ảnh bằng cách đơn giản là mở chúng trong một trình soạn thảo văn bản với phông chữ có độ rộng cố định. Chúng ta sẽ phân loại các hình ảnh của các chữ số từ 0 đến 9, nghĩa là khi cho một hình ảnh của một con số, chúng ta phải xác định được chính xác đó là số nào.

Vì máy tính không thể trực tiếp hiểu được một hình ảnh, chúng ta cần phải mô tả dữ liệu đầu vào dưới dạng mà máy tính có thể hiểu được. Chúng ta làm điều này bằng cách lấy một hình ảnh đầu vào và trích xuất một tập hợp các đặc trưng từ nó.

Một đặc trưng đơn giản mô tả một cái gì đó về dữ liệu của chúng ta. Trong trường hợp hình ảnh, chúng ta có thể đơn giản chia hình ảnh thành một lưới các pixel và nói rằng mỗi pixel có một đặc trưng mô tả giá trị của nó.

Chúng ta sẽ sử dụng các hình ảnh này như một tập hợp các đặc trưng nhị phân để đưa vào bộ phân loại của chúng ta. Vì hình ảnh có kích thước 28x28 pixel, chúng ta sẽ có $28 \times 28 = 784$ đặc trưng cho mỗi hình ảnh đầu vào. Để tạo ra các đặc trưng nhị phân từ dữ liệu này, chúng ta sẽ coi pixel tại vị trí i, j là một đặc trưng và nói rằng $F_{i,j}$ có giá trị 0 nếu nó là pixel nền (màu trắng) và 1 nếu nó là pixel tiền cảnh (màu xám hoặc đen). Trong trường hợp này, chúng ta đang tổng quát hóa hai giá trị tiền cảnh là xám và đen thành một để đơn giản hóa bài toán.

2.7.4 Hướng giải quyết

Phân loại Naive Bayes dựa trên định lý Bayes, định lý này mô tả xác suất của một sự kiện dựa trên kiến thức trước đó về các điều kiện có thể liên quan đến sự kiện đó. Trong trường hợp của chúng ta, chúng ta đang tính toán xác suất mà một hình ảnh thuộc về một lớp dựa trên giá trị của một đặc trưng. Ý tưởng này được gọi là xác suất có điều kiện và là khả năng xảy ra sự kiện A khi biết sự kiện B đã xảy ra, được viết là $\mathbb{P}(A|B)$. Từ đó, chúng ta có định lý Bayes:

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(B|A)\mathbb{P}(A)}{\mathbb{P}(B)}$$

Điều này có nghĩa là nếu A và B là các sự kiện và $\mathbb{P}(B)$ khác 0 thì:

- $\mathbb{P}(A|B)$ là xác suất A xảy ra khi biết B đã xảy ra
- $\mathbb{P}(B|A)$ là xác suất B xảy ra khi biết A đã xảy ra
- $\mathbb{P}(A)$ là xác suất A xảy ra độc lập với B
- $\mathbb{P}(B)$ là xác suất B xảy ra độc lập với A

Áp dụng vào trường hợp của chúng ta, chúng ta muốn tính toán cho mỗi đặc trưng, xác suất rằng khi đặc trưng F_{ij} có giá trị $f \in \{0, 1\}$ thì hình ảnh sẽ thuộc vào một lớp $c \in \{0, 1, \dots, 9\}$. Để làm điều này, chúng ta sẽ huấn luyện mô hình của mình trên một tập hợp lớn các hình ảnh mà chúng ta biết lớp chúng thuộc về. Sau khi tính được các xác suất này, chúng ta sẽ có thể phân loại hình ảnh bằng cách tính lớp có xác suất cao nhất cho tất cả các đặc trưng đã biết của hình ảnh và gán hình ảnh đó vào lớp đó.

Từ đó, chúng ta có được hai giai đoạn của thuật toán:

- Huấn luyện: Tính toán xác suất có điều kiện cho mỗi đặc trưng của một hình ảnh thuộc về mỗi lớp.
- Phân loại: Đối với mỗi hình ảnh, tính toán lớp mà hình ảnh đó có khả năng

thuộc vào nhất, dựa trên các xác suất độc lập đã tính được ở giai đoạn huấn luyện.

2.7.5 Huấn luyện

Mục tiêu của giai đoạn huấn luyện là dạy cho máy tính xác suất để một đặc trưng $F_{i,j}$ có giá trị $f \in \{0, 1\}$ (vì chúng ta có các đặc trưng nhị phân), với điều kiện là hình ảnh hiện tại thuộc lớp $c \in \{0, 1, \dots, 9\}$. mà chúng ta có thể viết là $\mathbb{P}(F_{i,j} = f \mid \text{class} = c)$.

Điều này có thể tính đơn giản như sau:

$$\mathbb{P}(F_{i,j} = f \mid \text{class} = c) = \frac{\# \text{ số lần } F_{i,j} = f \text{ khi class} = c}{\text{Tổng số ví dụ huấn luyện mà class} = c} \quad (2.19)$$

Vì cách chúng ta sẽ sử dụng các xác suất này, chúng ta cần đảm bảo rằng chúng không bằng không. Nếu bằng không, chúng sẽ triệt tiêu bất kỳ xác suất khác không nào từ các đặc trưng khác. Để làm điều này, chúng ta sử dụng một kỹ thuật gọi là Laplace Smoothing. Kỹ thuật này hoạt động bằng cách cộng thêm một giá trị dương nhỏ k vào tử số ở trên và $k.V$ vào mẫu số (trong đó V là số giá trị có thể mà đặc trưng của chúng ta có thể nhận, trong trường hợp này là 2). Giá trị k càng lớn thì quá trình làm mịn càng mạnh. Vì vậy, đối với các đặc trưng nhị phân của chúng ta, ta sẽ có:

$$\mathbb{P}(F_{i,j} = f \mid \text{class} = c) = \frac{k + \# \text{ số lần } F_{i,j} = f \text{ khi class} = c}{2k + \text{Tổng số ví dụ huấn luyện mà class} = c} \quad (2.20)$$

Bạn có thể thử nghiệm với các giá trị k khác nhau (ví dụ từ 0.1 đến 10) và tìm ra giá trị cho độ chính xác phân loại cao nhất.

Bạn cũng nên ước tính xác suất trước $\mathbb{P}(\text{class} = c)$ hoặc xác suất của mỗi lớp độc lập với các đặc trưng bằng tần suất thực nghiệm của các lớp khác nhau trong tập huấn luyện.

$$P(\text{class} = c) = \frac{\# \text{ Số ví dụ huấn luyện mà class} = c}{\# \text{ Tổng số ví dụ huấn luyện}} \quad (2.21)$$

Sau khi hoàn thành quá trình huấn luyện, bạn sẽ có một tập hợp các xác suất được tạo ra một cách kinh nghiệm, có thể được gọi là mô hình mà chúng ta sẽ sử dụng để phân loại dữ liệu chưa biết. Mô hình này một khi đã được tạo ra có thể được sử dụng trong tương lai mà không cần phải tạo lại.

2.7.6 Phân lớp

Để phân loại những hình ảnh chưa biết bằng mô hình đã được huấn luyện, chúng ta sẽ thực hiện phân loại xác suất hậu nghiệm cực đại (MAP) đối với dữ liệu kiểm tra bằng mô hình đã huấn luyện. Kỹ thuật này tính toán xác suất hậu nghiệm của mỗi lớp cho một hình ảnh cụ thể. Sau đó, hình ảnh sẽ được phân vào lớp có xác suất hậu nghiệm cao nhất.

Bởi vì chúng ta giả định rằng tất cả các xác suất đều độc lập với nhau, chúng ta có thể tính xác suất mà hình ảnh thuộc về một lớp bằng cách đơn giản nhân tất cả các xác suất lại với nhau và chia cho xác suất của tập hợp các đặc trưng. Cuối cùng, vì chúng ta không thực sự cần giá trị xác suất mà chỉ cần so sánh các giá trị, chúng ta có thể bỏ qua xác suất của tập hợp các đặc trưng và do đó tính toán giá trị như sau:

$$\mathbb{P}(class) \times \mathbb{P}(f_{1,1}|class) \times \mathbb{P}(f_{1,2}|class) \times \dots \times \mathbb{P}(f_{28,28}|class) \quad (2.22)$$

Thật không may, khi máy tính thực hiện phép tính số thực dấu chấm động, chúng không thực hiện chính xác như cách chúng ta làm bằng tay và có thể dẫn đến một loại lỗi gọi là tràn số. Bạn có thể tìm thêm thông tin về lỗi này tại đây:

[Dòng chảy số học](#)

Để tránh vấn đề này, chúng ta sẽ tính toán bằng cách sử dụng logarit của mỗi xác suất thay vì giá trị thực tế. Bằng cách này, phép tính thực tế sẽ như sau:

$$\log \mathbb{P}(class) + \log \mathbb{P}(f_{1,1}|class) + \log \mathbb{P}(f_{1,2}|class) + \dots + \log \mathbb{P}(f_{28,28}|class) \quad (2.23)$$

Trong đó, F_{ij} là giá trị đặc trưng của dữ liệu đầu vào. Bạn nên sử dụng các xác suất trước (prior) và xác suất có điều kiện (probabilities) đã tính toán được trong giai đoạn huấn luyện để tính toán các xác suất hậu nghiệm này.

Một khi bạn đã có xác suất hậu nghiệm cho mỗi lớp từ 0 đến 9, bạn nên gán dữ liệu đầu vào cần kiểm tra vào lớp có xác suất hậu nghiệm cao nhất. Ví dụ, đối với một dữ liệu đầu vào P, nếu chúng ta có các xác suất hậu nghiệm như sau:

class	xác suất hậu nghiệm
0	0.3141
1	0.432
2	0
3	0
4	0.4
5	0.004
6	0.1
7	0.2
8	0.7
9	0.5

Chúng ta phân loại 'P' là số 8 vì đó là lớp có xác

suất hậu nghiệm cao nhất.

2.7.7 Yêu cầu

Để hoàn thành bài tập này, bạn sẽ cần sử dụng Python để viết một chương trình áp dụng thuật toán Naive Bayes (đã được mô tả ở trên) để phân loại các hình ảnh được cung cấp. Chương trình này phải chạy được trên Google Colab và có các chức năng sau:

- Đọc dữ liệu huấn luyện từ các file và tạo một mô hình: Đọc dữ liệu hình ảnh từ các file và xây dựng một mô hình Naive Bayes dựa trên dữ liệu đó.
- Lưu mô hình thành một file: Lưu mô hình đã được huấn luyện vào một file để sử dụng sau này.
- Đọc mô hình từ một file: Đọc một mô hình đã được lưu trước đó từ một file.
- Phân loại hình ảnh từ một file: Sử dụng mô hình đã huấn luyện để phân loại các hình ảnh mới.

Trước khi bắt đầu viết code, bạn nên lên kế hoạch chi tiết cho chương trình của mình, bao gồm việc chia nhỏ chương trình thành các hàm con và xác định các biến cần thiết để lưu trữ thông tin.

Sau khi hoàn thành mô hình, bạn cần đánh giá hiệu suất của nó bằng cách so sánh kết quả phân loại với nhãn thực tế của dữ liệu.

a, Đánh giá mô hình

Sử dụng nhãn lớp thực tế của các hình ảnh kiểm tra từ tệp testlabels để kiểm tra độ chính xác của mô hình của bạn. Báo cáo ma trận nhầm lẫn của bạn. Đây là một ma trận 10x10, trong đó phần tử ở hàng thứ r và cột thứ c là tỷ lệ hình ảnh kiểm tra từ lớp r được phân loại là lớp c .

Trong trường hợp lý tưởng, ma trận này sẽ có giá trị 1 trên đường chéo và 0 ở các vị trí khác, điều này xảy ra khi mọi hình ảnh đều được phân loại chính xác. Tuy nhiên, điều này sẽ không xảy ra với bộ phân loại của chúng ta vì chúng chưa đủ tốt. Không có một câu trả lời chính xác duy nhất cho phần này, nhưng nó sẽ giúp bạn hiểu được hiệu suất của bộ phân loại.

Nếu bạn có thời gian, bạn cũng có thể thử các cách sau:

- Điều chỉnh hệ số làm mịn Laplace: Thử nghiệm với các giá trị khác nhau của hệ số làm mịn Laplace để tìm ra cách giúp bộ phân loại của bạn đạt hiệu suất tốt nhất.
- Thay đổi thứ tự dữ liệu huấn luyện: Thử ngẫu nhiên thứ tự dữ liệu huấn luyện để xem liệu điều đó có ảnh hưởng đến kết quả của bộ phân loại hay không.

b, Yêu cầu phụ

- Thực hiện kiểm định chéo K-fold với các giá trị K khác nhau để cải thiện độ chính xác phân loại.
- Sử dụng các đặc trưng ba trạng thái (bằng cách xem xét hai giá trị nền trước) để xem liệu độ chính xác phân loại có cải thiện hay không.
- Sử dụng các nhóm pixel làm đặc trưng, ví dụ như các ô vuông 2x2 pixel, trong đó giá trị của một đặc trưng là giá trị phổ biến nhất của các pixel cấu thành nên nó.

c, Bảng thuật ngữ

Chúng tôi nhận thấy bạn đã được giới thiệu với nhiều thuật ngữ và khái niệm toán học, vì vậy chúng tôi đã tổng hợp một bảng thuật ngữ để bạn tham khảo:

Xác suất $\mathbb{P}(X)$: Là xác suất xảy ra sự kiện X, có giá trị từ 0 đến 1.

Xác suất có điều kiện $\mathbb{P}(A|B)$: Là xác suất xảy ra sự kiện A khi biết rằng sự kiện B đã xảy ra.

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$$

Định lý Bayes: Dự đoán xác suất của một sự kiện dựa trên kiến thức đã có.

$$\mathbb{P}(C|X) = \frac{\mathbb{P}(C) \times \mathbb{P}(X|C)}{\mathbb{P}(X)}$$

Đặc trưng: Chúng ta sử dụng các đặc trưng để mô tả một đặc tính của dữ liệu đầu vào. Một tập hợp đặc trưng là tập hợp tất cả các đặc trưng mô tả một phần dữ liệu đầu vào. Trong trường hợp này, chúng ta có 1 đặc trưng cho mỗi pixel trong một ảnh 28x28, tức là chúng ta có 784 đặc trưng.

Lớp: Lớp mô tả nhãn của một phần dữ liệu. Trong trường hợp này, lớp của chúng

ta có thể nhận bất kỳ giá trị nào từ 0-9.

Làm mịn Laplace: Cộng thêm k vào tử số trong $\mathbb{P}(F_{ij} = f | \text{class} = c)$ và $k.V$ (với V là số giá trị mà đặc trưng có thể nhận) vào mẫu số. Điều này đảm bảo không có số đếm nào bằng không và xác suất hậu nghiệm không bằng không.

Phân loại MAP: Được sử dụng để phân loại nhãn của một đầu vào kiểm tra. Chúng ta tính toán xác suất hậu nghiệm của tất cả các lớp và gán lớp có xác suất hậu nghiệm cao nhất cho đầu vào của chúng ta.

Ma trận nhầm lẫn: Một ma trận 10×10 , trong đó phần tử ở hàng R và cột C cho biết tỷ lệ hình ảnh kiểm tra từ lớp R được phân loại là lớp C .

Mô hình: Một mô hình trong học máy là kết quả của quá trình huấn luyện trên một tập dữ liệu. Trong trường hợp này, mô hình là tập hợp các xác suất cho mỗi đặc trưng và mỗi lớp.

2.7.8 Lời giải

a, Cài đặt code

- Thêm các thư viện cần thiết : numpy, matplotlib, pandas.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

- Ta đọc dữ liệu hình ảnh 28×28 đầu vào và chuyển đổi các kí tự trắng về '0' và các kí tự còn lại về '1':

```
# Đọc dữ liệu từ đường dẫn chứa file
images = []
with open(file_path, 'r') as f:
    lines = f.readlines() # cần đầu
vào là đường dẫn file

# Dùng lệnh lặp để đọc tất cả dòng file hình
ảnh rồi lưu trữ vào danh sách
current_image = []
for line in lines:
```

```

        if line == '\n':
            continue
        # Chuyển đổi dòng sang format số (' ' ->
0, '+' -> 1, '#' -> 1)
        current_image.extend([1 if char == '+' or
char == '#' else 0 for char in line[:28]])
        # Khi tích tụ đủ 28 dòng (1 ảnh) thì đẩy
vào danh sách images
        if len(current_image) == 28 * 28:
            images.append(current_image)
            current_image = []

# Chuyển danh sách thành một dãy numpy
X = np.array(images, dtype=np.int32)
return X                                # đầu ra sẽ là một dãy

```

- Ta đọc các nhãn, rồi chuyển nó thành một dãy numpy:

```

def load_labels(label_file_path):
    with open(label_file_path, 'r') as f:
        labels = [int(line.strip()) for line in f]
    return np.array(labels, dtype=np.int32).reshape(-
1)

```

```

# Function to load and preprocess images
def load_images(file_path):
    """
    Load and preprocess images from a file.

    Parameters:
        file_path: str, path to the file containing image data

    Returns:
        X: np.ndarray, shape (num_samples, 784), binary pixel data
    """
    images = []
    with open(file_path, 'r') as f:
        lines = f.readlines()
    current_image = []
    for line in lines:
        if line == '\n':
            continue
        # Convert the line to numerical format (' ' -> 0, '+' -> 1, '#' -> 1)
        current_image.extend([1 if char == '+' or char == '#' else 0 for char in line[:28]])
        # If we've accumulated 28 lines (one image), add it to the images list
        if len(current_image) == 28 * 28:
            images.append(current_image)
            current_image = []
    # Convert images list to numpy array
    X = np.array(images, dtype=np.int32)
    return X

# Function to load labels
def load_labels(label_file_path):
    """
    Load labels from a file.

    Parameters:
        label_file_path: str, path to the file containing labels

    Returns:
        y: np.ndarray, shape (num_samples,), labels for the images
    """
    with open(label_file_path, 'r') as f:
        labels = [int(line.strip()) for line in f]
    return np.array(labels, dtype=np.int32).reshape(-1)

```

- Kiểm tra dữ liệu đầu vào sau khi đọc:

```
def view(X, idx):
    for i in range(28):
        for j in range(28):
            print(X[idx, i, j], end='')
        print()

view(X_test, 0)
```

- Sau khi ta chạy lệnh, kết quả ta có được là ảnh cỡ 28x28 thứ nhất.



b, Cài đặt hàm

- Ta cài đặt hàm Naive Bayes với những tham số đầu vào sau:
 - self.num_classes: Số lượng class trong tập dữ liệu
 - self.num_features: Số lượng các đặc điểm mỗi đặc trưng
 - self.priors: Khởi tạo trước một zeros list của $\mathbb{P}(i)$
 - self.likelihoods: Ma trận chứa $\mathbb{P}(F_i = f|c)$ với c thuộc $0, 1, \dots, 9$ và i thuộc $0, 1, \dots, 783$
 - Hàm tính $\mathbb{P}(c)$ và $\mathbb{P}(F_i = 1|c)$ từ tập dữ liệu training tương ứng với công

thức (Ở đây chúng ta chỉ cần tính $\mathbb{P}(F_i = 1|c)$ vì

$$\mathbb{P}(F_i = 0|c) = 1 - \mathbb{P}(F_i = 1|c)$$

```
class NaiveBayes:
    priors = None
    likelihoods = None

    def __init__(self, X, y):
        # Initialize the number of classes and features
        self.num_classes = X.shape[0]
        self.num_features = X.shape[1]
        # Initialize priors and likelihoods arrays
        self.priors = np.zeros(self.num_classes)
        self.likelihoods = np.zeros((self.num_classes, self.num_features))
        # Get unique class labels
        self.classes = np.unique(y)

    def fit(self, X, y):
        # Number of unique classes
        n_classes = len(self.classes)
        # Laplace smoothing parameter
        alpha = 0.1
        for idx, cols in enumerate(self.classes):
            # Select samples belonging to the current class
            X_cols = X[y == cols]
            # Calculate prior probability for the current class
            self.priors[idx] = X_cols.shape[0] / float(self.num_classes)
            # Calculate feature counts for the current class with Laplace smoothing
            class_feature_count = X_cols.sum(axis=0) + alpha
            total_feature_count = X_cols.sum() + alpha * self.num_features
            # Calculate likelihoods for the current class
            self.likelihoods[idx, :] = class_feature_count / total_feature_count
```

- Cuối cùng chúng ta áp dụng công thức đã tìm được ở trên để dự đoán ra các class tương ứng của test data:

```
def predict_helper(self, x):
    posteriors = []
    for idx, cols in enumerate(self.classes):
        # Calculate the log prior probability
        prior = np.log(self.priors[idx])
        # Calculate the log likelihood for the given sample
        likelihood = np.sum(x * np.log(self.likelihoods[idx]))
        # Calculate the posterior probability
        posterior = prior + likelihood
        posteriors.append(posterior)
    # Return the class with the highest posterior probability
    return self.classes[np.argmax(posteriors)]

def predict(self, X):
    # Predict the class for each sample in X
    y_pred = np.array([self.predict_helper(x) for x in X])
    return y_pred

def accuracy(self, y_true, y_pred):
    # Calculate the accuracy of the predictions
    return np.mean(y_true == y_pred)
```

- Code cho toàn bộ hàm:

```
class NaiveBayes:
    priors = None
    likelihoods = None

    # Hàm khởi tạo
```

```

def __init__(self, X, y):
    # Khởi tạo số lớp và số thuộc tính
    self.num_classes = X.shape[0]
    self.num_features = X.shape[1]
    # Khởi tạo xác suất priori và xác suất có điều
    kiện
    self.priors = np.zeros(self.num_classes)
    self.likelihoods = np.zeros((self.num_classes,
self.num_features))
    # Lưu các nhãn duy nhất
    self.classes = np.unique(y)

# Hàm huấn luyện
def fit(self, X, y):
    # Xuất ra số lượng nhãn
    n_classes = len(self.classes)
    # Tham số làm mượt alpha
    alpha = 0.1
    for idx, cols in enumerate(self.classes):
        # Chọn các mẫu trong X tương đương với cols
        X_cols = X[y == cols]
        # Tính xác suất prior cho lớp hiện tại
        self.priors[idx] = X_cols.shape[0] / float(self.n
        # Đếm số lượng thuộc tính cho lớp hiện tại
        với hệ số alpha
        class_feature_count = X_cols.sum(axis=0)
        + alpha
        total_feature_count = X_cols.sum() + alpha
        * self.num_features
        # Tính xác suất điều kiện của mỗi đặc trưng
        cho lớp hiện tại
        self.likelihoods[idx, :] = class_feature_count
        / total_feature_count

# Hàm tính xác suất hậu nghiệm
def predict_helper(self, x):
    posteriors = []

```

```

        for idx, cols in enumerate(self.classes):
            # Tính xác suất prior bằng logarit
            prior = np.log(self.priors[idx])
            # Tính xác suất điều kiện cho mẫu hiện tại
            likelihood = np.sum(x * np.log(self.likelihoods[idx]))
            # Tính xác suất hậu nghiệm
            posterior = prior + likelihood
            posteriors.append(posterior)
        # Trả về lớp có xác suất hậu nghiệm cao nhất
        return self.classes[np.argmax(posteriors)]

# Hàm dự đoán, sử dụng hàm predict_helper làm đầu vào
def predict(self, X):
    # Dự đoán lớp cho X
    y_pred = np.array([self.predict_helper(x) for
x in X])
    return y_pred

# Hàm tính toán độ chính xác
def accuracy(self, y_true, y_pred):
    # Tính toán độ chính xác trong tính toán
    return np.mean(y_true == y_pred)

```

c, Kết quả thử nghiệm

- Ta áp dụng mô hình và có kết quả thử nghiệm như sau

```

model = NaiveBayes(X_train, y_train)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print('Accuracy: ' + str(model.accuracy(y_test, y_pred)))
✓ 0.1s
Accuracy: 0.757

```

- Code cho phần kết quả:

```

model = NaiveBayes(X_train, y_train)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(model.accuracy(y_test, y_pred))

```


2.8 Tài liệu tham khảo

- Thuật toán phân lớp Naive Bayes | VILBO
- Naive Bayes Classifier | Machine Learning cơ bản (machinelearningcoban.com)

K-MEANS

Có hai người bạn thân tên Khánh và Trung dự định sẽ cùng mua một món hàng đặc biệt trên Shopee cho dự án riêng của họ, và tình cờ họ có phát hiện ra một điều này. Bạn Trung lúc đặt hàng trên đây và sau khi áp mã giảm giá thấy giá rẻ bất thường, nên bảo bạn Khánh áp thử tất cả những mã giảm giá mà bạn ấy có, và bạn Khánh thấy giá thanh toán sau khi áp dụng voucher bên mình cao hơn khá nhiều so với giá thanh toán sau khi bạn Trung áp vào. Cả 2 người có ngồi tranh luận thì bạn Trung có nói: "Tao có một số mã giảm giá khá đặc biệt, được Shopee ưu đãi luôn nhé, hay có khi do tao mua hàng nhiều nhỉ, cái mã này chỉ được áp dụng cho đối tượng nhất định". Khánh ngẫm nghĩ, có khi đúng thật, tại mình mua hàng trên Shopee không nhiều.

Đây hoàn toàn là câu chuyện có thật, và nó cũng đặt ra vấn đề rằng: Bạn Trung mua nhiều, và bạn ấy được hưởng rất nhiều ưu đãi đặc biệt và có giới hạn, mua càng nhiều thì chắc chắn càng được hưởng nhiều chính sách đặc quyền của Shopee hơn nữa. Đó là một trong số những ứng dụng rộng rãi của một thuật toán cực kỳ nổi tiếng trong lĩnh vực Học máy - Machine Learning mà chúng em sẽ làm rõ ở bài tập lớn này. Chúng em xin được giới thiệu, thuật toán phân cụm K-means.

Thuật toán K-means hoạt động dựa trên nguyên tắc của việc phân cụm dữ liệu thành các nhóm (clusters) sao cho các điểm dữ liệu trong cùng một nhóm có sự tương đồng cao và các điểm dữ liệu khác nhóm có sự khác biệt lớn. Điểm mạnh của thuật toán K-means là tính đơn giản và hiệu suất tính toán nhanh. Thuật toán này có thể được triển khai một cách dễ dàng và đạt hiệu suất tốt trên các tập dữ liệu lớn.

3.1 Giới thiệu

Ra đời từ năm 1957, thuật toán K-means của Lloyd K-means đã trở thành một trong những phương pháp phân cụm kinh điển và được ứng dụng rộng rãi nhất. Nguyên lý hoạt động của K-means dựa trên việc phân chia dữ liệu thành các nhóm (cụm) riêng biệt.

K-Means là một thuật toán học không giám sát (unsupervised learning), tức là nó không cần các nhãn đúng cho dữ liệu đầu vào. Thuật toán này được sử dụng phổ biến trong việc phân cụm dữ liệu, nghĩa là chia tập dữ liệu thành các nhóm (clusters) dựa trên sự tương đồng giữa các điểm dữ liệu.

Thuật toán K-means, với giả định về các cụm hình cầu và phân bố dữ liệu đồng nhất, thường được xem như một công cụ phân cụm đơn giản và hiệu quả. Tuy nhiên,

khi đối mặt với dữ liệu thực tế có cấu trúc phức tạp hơn, chẳng hạn như dữ liệu có hình dạng bất thường, độ đậm đặc khác nhau hoặc chứa nhiễu, giả định này trở nên quá cứng nhắc. Dù vậy, K-means vẫn là một thuật toán khởi điểm tốt và được sử dụng rộng rãi nhờ tính dễ hiểu và khả năng xử lý lượng lớn dữ liệu.

3.2 Thuật toán K-means

Đầu tiên ta sẽ xét biểu diễn dữ liệu: Giả sử tập dữ liệu $D = \{x_1, x_2, \dots, x_n\}$ trong đó mỗi x_i là một vector n chiều trong không gian Euclid. Phương pháp K-means phân nhóm tập dữ liệu D thành K cụm khác nhau với:

- Mỗi cụm có một điểm trung tâm, được gọi là centroid.
- Số lượng cụm K là một giá trị cố định được chọn trước.

3.2.1 Thuật toán này hoạt động như nào?

Đầu vào: Cho tập dữ liệu D , với K là số cụm, phép đo khoảng cách giữa 2 điểm dữ liệu là $d(x,y)$. Giả sử chúng ta muốn chia các điểm sau thành các cụm.



Hình 3.1: Các điểm mà ta muốn phân chia thành cụm.

Khởi tạo: Khởi tạo K điểm dữ liệu trong D làm các điểm trung tâm (centroid). Đầu tiên, chúng ta phải chọn bao nhiêu cụm mà chúng ta muốn phân? K trong ‘K-means’ là viết tắt của số cụm mà chúng ta đang muốn xác định. Tại bước này thuật toán sẽ khởi tạo k điểm dữ liệu trung tâm ban đầu, sẽ bắt đầu với việc chọn 2 cụm, đối với 3, 4 cụm thuật toán cũng sẽ làm tương tự.

Bước thứ 2 là chỉ định 1 điểm, chúng em sẽ gọi nó là hạt trung tâm để bắt đầu (nó được chọn ngẫu nhiên hoặc cũng được chỉ định bởi các nhà khoa học dữ liệu dựa trên kiến thức trước đó về dữ liệu). Phân thành 2 cụm sẽ có 2 hạt trung tâm, ở đây là hạt màu xanh lá cây và màu cam như hình dưới.



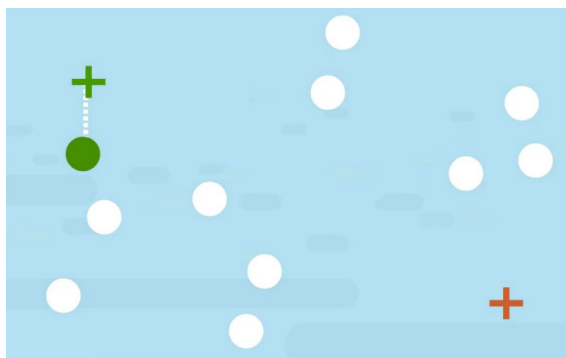
Hình 3.2: Hai hạt trung tâm đã được lựa chọn ngẫu nhiên là màu xanh lá cây và màu cam để thực hiện thuật toán.

Để xác định điểm trung tâm của cluster ta sử dụng công thức như sau:

$$m_i = \frac{1}{|C_i|} \sum_{x \in C_i} x \quad (3.1)$$

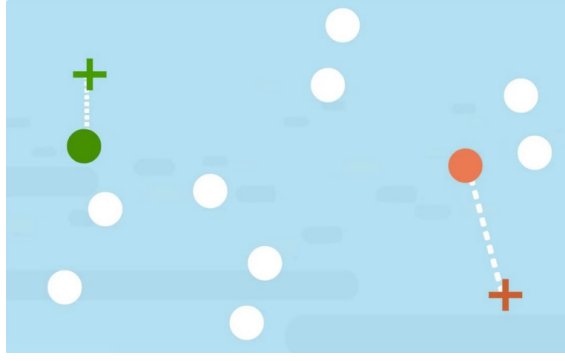
Trong đó C_i là cluster thứ i , m_i là điểm trung tâm của cluster C_i tương ứng.

Bước tiếp là tính khoảng cách của hạt trung tâm với các hạt khác. Nếu điểm nào gần trung tâm hơn nó sẽ được gán là màu của hạt trung tâm đó. Ví dụ điểm này gần với hạt màu xanh lá cây hơn là điểm màu cam hơn nên nó sẽ thuộc cụm xanh lá.



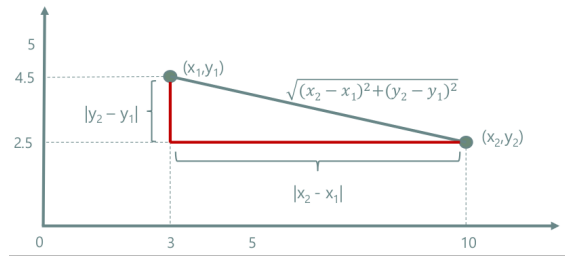
Hình 3.3: Cụm xanh lá đã chọn được điểm gần nhất.

Mặt khác, điểm gần với hạt màu cam hơn nên nó sẽ là 1 phần của cụm cam.



Hình 3.4: Kế tiếp, cụm màu cam cũng đã được chọn điểm gần nhất trong tất cả các điểm.

Bằng cách này chúng ta có thể gán tất cả các điểm trên biểu đồ trong mỗi cụm, dựa trên tính khoảng cách giữa 2 điểm biết tọa độ. Trong K-means để đánh giá mức độ giống nhau hay khoảng cách giữa 2 điểm dữ liệu ta có thể sử dụng các phép đo khoảng cách khác nhau. Dưới đây là sơ đồ tổng quát của công thức khoảng cách Euclidean:



Hình 3.5: Sơ đồ tổng quát của công thức khoảng cách Euclidean.

Từ đó ta có công thức tính sau:

$$d(\mathbf{x}, \mathbf{m}_i) = \|\mathbf{x} - \mathbf{m}_i\| = \sqrt{(x_1 - m_{i1})^2 + (x_2 - m_{i2})^2 + \dots + (x_n - m_{in})^2} \quad (3.2)$$

Tiếp tục với biểu đồ phân lớp bên trên sau khi đã gán hết cụm của các điểm, chúng ta sẽ tính lại tọa độ hạt trung tâm thêm lần nữa rồi lặp lại các bước tính khoảng cách trên để gán lại các điểm.



Hình 3.6: Các điểm sau khi đã được gán nhãn phù hợp.

Chúng ta sẽ tính tọa độ hạt trung tâm mới thông qua công thức sau đây:

$$m_{ij} = \frac{1}{|C_i|} \sum_{x \in C_i} x_j \quad (3.3)$$

Trong đó C_i là cluster thứ i , m_{ij} là điểm trung tâm mới của cluster C_i tương ứng. Còn x_j là giá trị tọa độ của điểm x trên chiều j .

Chúng ta lặp lại sau khoảng 2 lần 3 lần hoặc có thể 1000 lần đến khi nào thuật toán hội tụ thì dừng lại.



Hình 3.7: Kết quả cuối cùng của thuật toán phân cụm K-means.

Nhân nói tới thuật toán hội tụ, khi nào ta biết thuật toán hội tụ? Ta sẽ xác định điều kiện dừng thuật toán theo một số cách như sau:

- Số điểm DL được gán lại nhỏ hơn một ngưỡng.
- Số centroid bị thay đổi nhỏ hơn một ngưỡng.
- Tổng bình phương lỗi nhỏ hơn một ngưỡng.

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} d(x, m_i)^2 \quad (3.4)$$

Trong đó C_i là cluster thứ i , m_i là điểm trung tâm của cluster C_i tương ứng.

Nhìn chung về điều kiện hội tụ có thể thấy mối liên hệ giữa các điều kiện là gần tương đồng như nhau. Khi có ít điểm dữ liệu được gán sang cluster khác có thể khiến điểm trung tâm không thay đổi nhiều và từ đó hàm mất mát cũng sẽ ít bị ảnh hưởng. Vậy nên ta có thể sử dụng 1 trong 3 cách trên để xác định điều kiện dừng của thuật toán.

3.2.2 Phương pháp Elbow

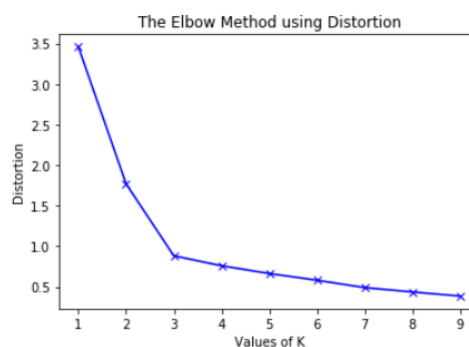
Để ý thấy rằng trong thuật toán nêu trên, chúng ta cần biết đại lượng K là số lượng cụm. Trong thực tế, nhiều trường hợp chúng ta không xác định được giá trị này. Trong bài tập lớn này, chúng em sẽ đưa ra một giải pháp để giải quyết vấn đề trên, đây là phương pháp Elbow - hay được biết đến với tên gọi Phương pháp Khuỷu tay. Phương pháp Elbow là một cách giúp ta lựa chọn được số lượng các cụm phù hợp dựa vào đồ thị trực quan hoá bằng cách nhìn vào sự suy giảm của hàm biến dạng và lựa chọn ra điểm khuỷu tay (elbow point).

Phương pháp này dựa trên khái niệm WCSS (Within-Cluster Sum of Squares). WCSS định nghĩa tổng biến thiên bên trong một cụm. Công thức tính WCSS chính là công thức (3.4)

Để tìm số cụm tối ưu, chúng ta sẽ thực hiện các bước sau:

- Khởi tạo số lượng cụm K .
- Thực hiện thuật toán K-means với số lượng cụm K đã chọn.
- Tính toán WCSS cho kết quả thu được từ thuật toán K-means.
- Tăng số lượng cụm K lên một giá trị mới và lặp lại bước 2 và bước 3.
- Đánh giá sự thay đổi của WCSS khi số lượng cụm tăng lên.
- Vẽ biểu đồ "elbow curve" với số lượng cụm trên trục x và WCSS trên trục y .
- Xác định điểm "khuỷu tay" trên biểu đồ, điểm này thường là điểm khi sự thay đổi của WCSS không còn đáng kể và giảm chậm.
- Số cụm tối ưu sẽ là số cụm tại điểm "khuỷu tay" trên biểu đồ.

Vì đồ thị cho thấy một khúc uốn cong sắc nét, trông giống như một khuỷu tay, nên nó được gọi là phương pháp khuỷu tay (Elbow method). Đồ thị của phương pháp Elbow trông giống như hình ảnh dưới đây:



Hình 3.8: Đồ thị của phương pháp Elbow:

3.3 Phân loại K-means

3.3.1 K-means

- Khởi tạo: Chọn ngẫu nhiên K trọng tâm cụm ban đầu từ dữ liệu.
- Gán điểm dữ liệu vào cụm gần nhất: Tính toán khoảng cách giữa từng điểm dữ liệu và các trọng tâm cụm, sau đó gán điểm dữ liệu vào cụm có trọng tâm gần nhất.
- Cập nhật trọng tâm cụm: Tính toán trung bình của các điểm dữ liệu trong mỗi cụm và cập nhật trọng tâm cụm.
- Lặp lại các bước gán và cập nhật cho đến khi không có sự thay đổi đáng kể trong vị trí trọng tâm cụm hoặc đạt được một số lượng lặp mà người dùng đã định trước.

3.3.2 K-means++

Một nhược điểm của thuật toán K-means là nó nhạy cảm với việc khởi tạo các trọng tâm hoặc các điểm trung bình. Vì vậy, nếu một centroid được khởi tạo là một điểm “xa”, nó có thể chỉ kết thúc không có điểm nào được liên kết với nó và đồng thời, nhiều hơn một cụm có thể kết thúc với một centroid duy nhất. Tương tự, nhiều hơn một trung tâm có thể được khởi tạo vào cùng một cụm dẫn đến phân cụm kém.

Để khắc phục vấn đề này, chúng ta đã có phương án phân cụm K-means++.

- Chọn ngẫu nhiên centroid đầu tiên từ các điểm dữ liệu.
- Đối với mỗi điểm dữ liệu, tính toán khoảng cách của nó từ trung tâm gần nhất, đã chọn trước đó.
- Chọn centroid tiếp theo từ các điểm dữ liệu sao cho xác suất chọn một điểm làm centroid tỷ lệ thuận với khoảng cách của nó từ centroid gần nhất, đã chọn trước đó. (tức là điểm có khoảng cách tối đa từ tâm gần nhất có nhiều khả năng được chọn tiếp theo làm tâm). Ta có thể làm tuần tự như sau: Lấy ngẫu nhiên điểm centroid đầu tiên m_1 , xong rồi Lấy điểm centroid tiếp theo là điểm xa nhất so với m_1 , cuối cùng là lấy điểm centroid thứ i (m_i) là điểm xa nhất so với $\{m_1, \dots, m_{i-1}\}$.
- Lặp lại các bước 2 và 3 cho đến khi k centroid được lấy mẫu.

3.3.3 K-means lô nhỏ

Với kích thước ngày càng tăng của các tập dữ liệu được phân tích, thời gian tính toán của K-means tăng lên do hạn chế của nó là cần toàn bộ tập dữ liệu trong bộ nhớ chính. Vì lý do này, một số phương pháp đã được đề xuất để giảm chi phí thời gian và không gian của thuật toán. Một cách tiếp cận khác là thuật toán K-means

lô nhỏ.

K-means lô nhỏ có nghĩa là phân cụm là một biến thể trên phân cụm K-means trong đó kích thước của tập dữ liệu đang được xem xét được giới hạn. Phân cụm K-mean bình thường hoạt động trên toàn bộ tập dữ liệu/đợt cùng một lúc, trong khi phân cụm K-mean theo lô nhỏ chia tập dữ liệu thành các tập con. Các lô nhỏ được lấy mẫu ngẫu nhiên từ toàn bộ tập dữ liệu và đối với mỗi lần lặp mới, một mẫu ngẫu nhiên mới được chọn và sử dụng để cập nhật vị trí của các trọng tâm.

Trong phân cụm K-Means theo lô nhỏ, các cụm được cập nhật với sự kết hợp của các giá trị lô nhỏ và tốc độ học. Tỷ lệ học tập giảm dần qua các lần lặp lại và tỷ lệ này là nghịch đảo của số lượng điểm dữ liệu được đặt trong một cụm cụ thể. Hiệu quả của việc giảm tốc độ học tập là tác động của dữ liệu mới bị giảm và đạt được sự hội tụ khi sau một vài lần lặp lại, không có thay đổi nào trong các cụm.

Kết quả nghiên cứu về hiệu quả của phân cụm K-means theo lô nhỏ cho thấy rằng nó có thể giảm thành công thời gian tính toán với sự đánh đổi nhẹ về chất lượng cụm.

- Khởi tạo: Chọn ngẫu nhiên K trọng tâm cụm ban đầu từ dữ liệu.
- Lặp lại cho đến khi đạt được số lần lặp tối đa.
- Chọn ngẫu nhiên một lô nhỏ từ dữ liệu.
- Gán điểm dữ liệu trong lô đây vào cụm gần nhất.
- Cập nhật trọng tâm cụm bằng cách tính toán trung bình của các điểm dữ liệu đã gán trong lô đây.
- Tiếp tục các bước gán và cập nhật giống như K-means.

3.3.4 K-medoids

K-Medoids là một thuật toán phân cụm không giám sát sử dụng phương pháp phân cụm. Đây là phiên bản ngẫu hứng của thuật toán phân cụm K-Means, đặc biệt được sử dụng để xử lý dữ liệu ngoại lệ. Nó yêu cầu dữ liệu không được gán nhãn để làm việc.

Trong thuật toán K-Medoids, mỗi điểm dữ liệu được gọi là medoid. Các medoids đóng vai trò là trung tâm cụm. Medoid là một điểm sao cho tổng khoảng cách của nó tới tất cả các điểm khác trong cùng một cụm là nhỏ nhất. Đối với khoảng cách, có thể sử dụng bất kỳ số liệu phù hợp nào như khoảng cách Euclidian hoặc khoảng cách Manhattan.

Dữ liệu hoàn chỉnh được chia thành K Cụm sau khi áp dụng thuật toán.

K-Medoids có ba loại – PAM, CLARA và CLARANS. PAM là phương pháp phổ biến nhất. Nó có một nhược điểm là mất nhiều thời gian.

K-Medoids được áp dụng theo cách mà

- Một điểm chỉ có thể thuộc về một cụm
- Mỗi cụm có tối thiểu một điểm

Chúng ta cùng xem K-Medoids hoạt động như nào nhé!

Ban đầu, chúng ta có K là số cụm và D là dữ liệu chưa được gán nhãn.

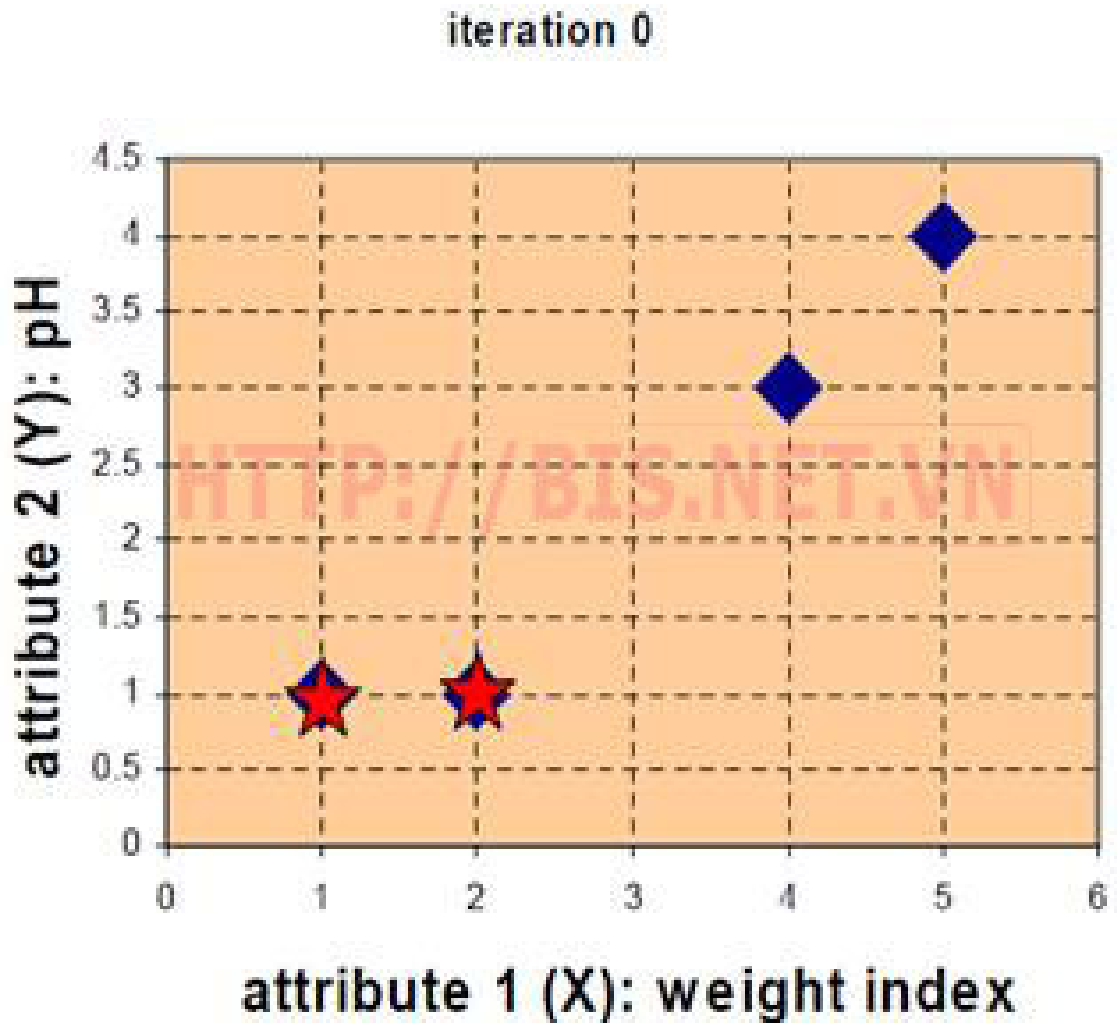
- Đầu tiên, chúng tôi chọn K điểm từ tập dữ liệu và gán chúng cho K số cụm. Những điểm K này đóng vai trò là medoids ban đầu. Mỗi đối tượng được đưa vào một cụm.
- Tiếp theo, khoảng cách giữa các medoid (điểm) ban đầu và các điểm khác (không phải medoid) được tính bằng cách sử dụng một trong các số liệu khoảng cách như khoảng cách Euclidian hoặc khoảng cách Manhattan,...
- Các điểm không phải medoid được gán cho cụm cụ thể đó mà khoảng cách của nó tới điểm medoid là nhỏ nhất
- Bây giờ tổng chi phí được tính là tổng khoảng cách từ các điểm khác đến điểm trung gian trong một cụm.
- Tiếp theo, một đối tượng non-medoid mới ngẫu nhiên s được chọn và được hoán đổi với đối tượng medoid ban đầu r , và chi phí được tính toán lại.
- Nếu chi phí $< cost_{tr}$ thì việc hoán đổi sẽ trở thành vĩnh viễn.
- Cuối cùng, các bước từ 2 đến 6 được lặp lại cho đến khi chi phí không thay đổi.

3.4 Ví dụ

Giả sử ta có 4 loại thuốc A, B, C, D , mỗi loại thuốc được biểu diễn bởi 2 đặc trưng X và Y như sau. Mục đích của ta là nhóm các thuốc đã cho vào 2 nhóm ($K=2$) dựa vào các đặc trưng của chúng.

Thuốc	chỉ số trọng lượng	pH
Thuốc A	1	1
Thuốc B	2	1
Thuốc C	4	3
Thuốc D	5	4

Bước 1. Khởi tạo tâm (centroid) cho 2 nhóm. Giả sử ta chọn A là tâm của nhóm thứ nhất (tọa độ tâm nhóm thứ nhất $c_1(1,1)$) và B là tâm của nhóm thứ 2 (tọa độ tâm nhóm thứ hai $c_2(2,1)$).



Biết được các thuộc của mỗi cụm, giờ đây chúng ta tính toán lại tâm của mỗi cụm dựa trên các thành viên mới này.

$$c_1 = \left(\frac{1+2}{2}, \frac{1+1}{2} \right) = \left(1\frac{1}{2}, 1 \right)$$

$$c_2 = \left(\frac{4+5}{2}, \frac{3+4}{2} \right) = \left(4\frac{1}{2}, 3\frac{1}{2} \right)$$

Bước 2. Tính khoảng cách từ các đối tượng đến tâm của các nhóm (Khoảng cách

Euclidean).

$$D^0 = \begin{bmatrix} 0 & 1 & 3.61 & 5 \\ 1 & 0 & 2.83 & 4.24 \end{bmatrix} \quad c_1 = (1, 1) \quad \text{group 1}$$

$$\begin{bmatrix} A & B & C & D \\ 1 & 2 & 4 & 5 & X \\ 1 & 1 & 3 & 4 & Y \end{bmatrix} \quad c_2 = (2, 1) \quad \text{group 2}$$

Mỗi cột trong ma trận khoảng cách D là một đối tượng (cột thứ nhất tương ứng với đối tượng A , cột thứ 2 tương ứng với đối tượng $B, \dots$). Hàng thứ nhất trong ma trận khoảng cách biểu diễn khoảng cách giữa các đối tượng đến tâm của nhóm thứ nhất c_1 và hàng thứ 2 trong ma trận khoảng cách biểu diễn khoảng cách của các đối tượng đến tâm của nhóm thứ 2 c_2 .

Ví dụ, khoảng cách từ loại thuốc $C=(4,3)$ đến tâm $c_1(1,1)$ là 3.61 và đến tâm $c_2(2,1)$ là 2.83 được tính như sau:

$$c_1 = (1, 1), \quad \sqrt{(4-1)^2 + (3-1)^2} = 3.61$$

$$c_2 = (2, 1), \quad \sqrt{(4-2)^2 + (3-1)^2} = 2.83$$

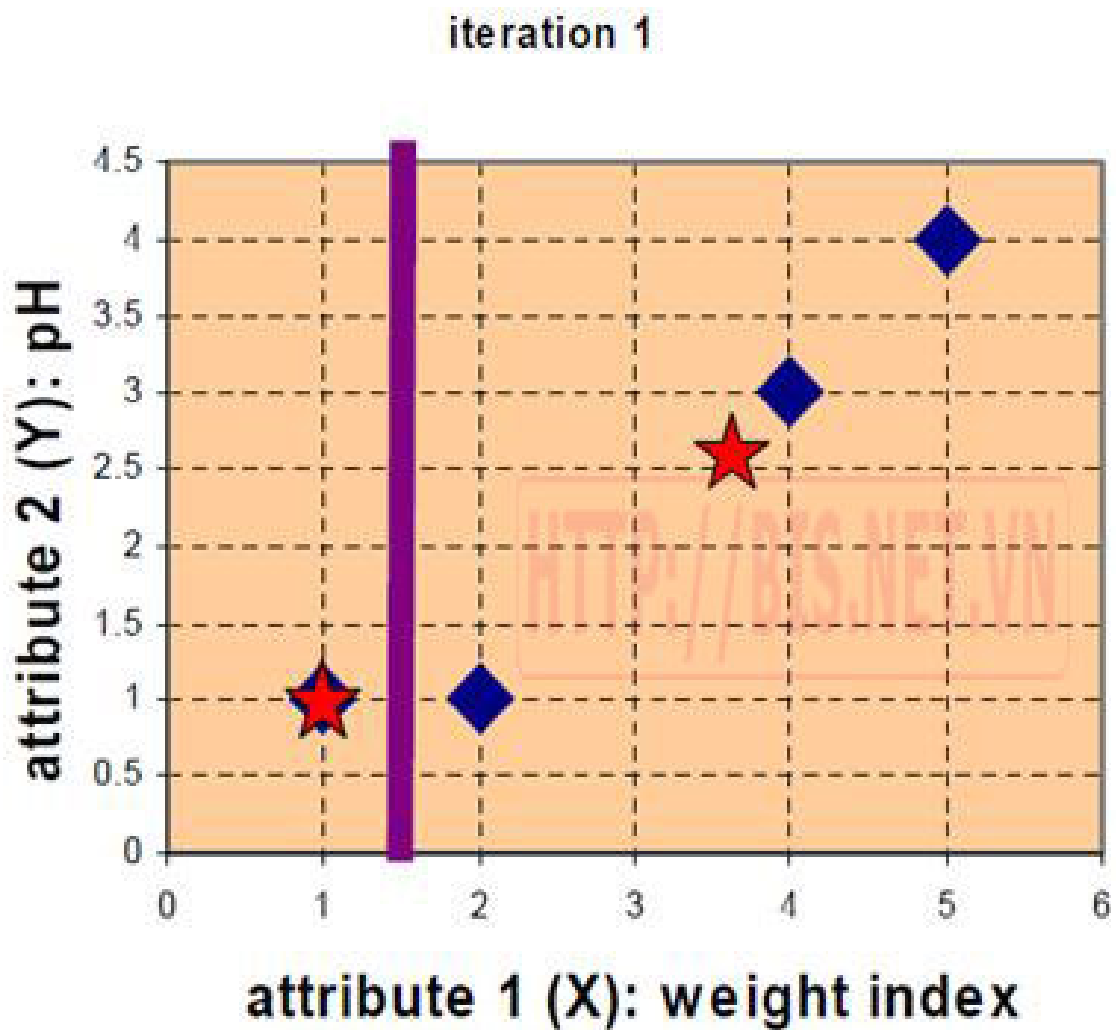
Bước 3. Nhóm các đối tượng vào nhóm gần nhất:

$$G^0 = \begin{bmatrix} A & B & C & D \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} \quad \begin{array}{l} \text{group-1} \\ \text{group-2} \end{array}$$

Ta thấy rằng nhóm 1 sau vòng lặp thứ nhất gồm có 1 đối tượng A và nhóm 2 gồm các đối tượng còn lại B, C, D .

Bước 4. Tính lại tọa độ các tâm cho các nhóm mới dựa vào tọa độ của các đối tượng trong nhóm. Nhóm 1 chỉ có 1 đối tượng A nên tâm nhóm 1 vẫn không đổi, $c_1(1,1)$. Tâm nhóm 2 được tính như sau:

$$c_2 = \left(\frac{2+4+5}{3}, \frac{1+3+4}{3} \right) = \left(\frac{11}{3}, \frac{8}{3} \right)$$



Bước 5. Tính lại khoảng cách từ các đối tượng đến tâm mới.

$$D^1 = \begin{bmatrix} 0 & 1 & 3.61 & 5 \\ 3.14 & 2.36 & 0.47 & 1.89 \end{bmatrix} \quad \begin{matrix} c_1 = (1, 1) & \text{group 1} \\ c_2 = (\frac{11}{3}, \frac{8}{3}) & \text{group 2} \end{matrix}$$

$$\begin{bmatrix} A & B & C & D \\ 1 & 2 & 4 & 5 & X \\ 1 & 1 & 3 & 4 & Y \end{bmatrix}$$

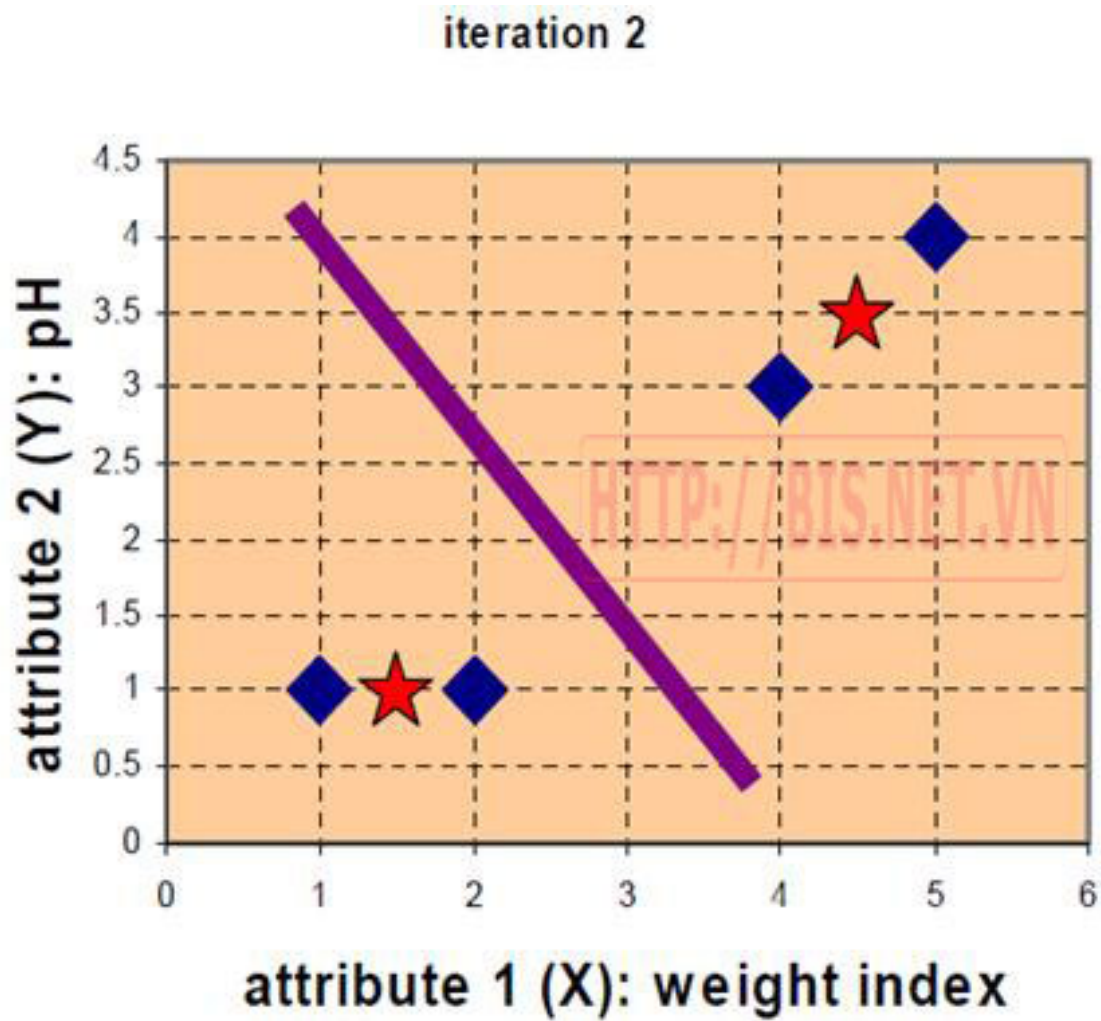
Bước 6. Nhóm các đối tượng vào nhóm.

$$G^1 = \begin{bmatrix} A & B & C & D \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \quad \begin{matrix} \text{group 1} \\ \text{group 2} \end{matrix}$$

Bước 7. Tính lại tâm cho nhóm mới.

$$c_1 = \left(\frac{1+2}{2}, \frac{1+1}{2} \right) = \left(1\frac{1}{2}, 1 \right)$$

$$c_2 = \left(\frac{4+5}{2}, \frac{3+4}{2} \right) = \left(4\frac{1}{2}, 3\frac{1}{2} \right)$$



Bước 8. Tính lại khoảng cách từ các đối tượng đến tâm mới.

$$D^2 = \begin{bmatrix} 0.5 & 0.5 & 3.2 & 4.61 \\ 4.3 & 3.54 & 0.71 & 0.71 \end{bmatrix} \quad c_1 = \left(1\frac{1}{2}, 1 \right) \quad \text{group 1}$$

$$\begin{bmatrix} A & B & C & D \\ 1 & 2 & 4 & 5 & X \\ 1 & 1 & 3 & 4 & Y \end{bmatrix} \quad c_2 = \left(4\frac{1}{2}, 3\frac{1}{2} \right) \quad \text{group 2}$$

Bước 9. Nhóm các đối tượng vào nhóm.

$$G^2 = \begin{bmatrix} A & B & C & D \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \quad \begin{array}{l} \text{group 1} \\ \text{group 2} \end{array}$$

Ta thấy $G^2 = G^1$ (Không có sự thay đổi nhóm nào của các đối tượng) nên thuật toán dừng và kết quả phân nhóm như sau:

Thuốc	chỉ số trọng lượng	pH	Nhóm
Thuốc A	1	1	1
Thuốc B	2	1	1
Thuốc C	4	3	2
Thuốc D	5	4	2

3.5 Ưu điểm và nhược điểm

Thuật toán K-means	Ưu điểm	Nhược điểm
	• Đơn giản và dễ hiểu. • Đảm bảo hội tụ trong thời gian đa thức. • Linh hoạt trong việc lựa chọn phương pháp đo khoảng cách. • Có thể mở rộng.	• Phụ thuộc vào số lượng cụm: K-means yêu cầu xác định trước số lượng cụm (clusters) cần phân loại. Điều này có thể là một thách thức khi không có thông tin rõ ràng về số lượng cụm cần tìm hoặc khi số lượng cụm thay đổi trong thời gian. Về cái này đã có phương án Elbow để giải quyết vấn đề trên. • Nhạy cảm với vị trí ban đầu: Kết quả của K-means có thể thay đổi do vị trí ban đầu của các centroid. Kết quả phân loại có thể khác nhau nếu chúng ta khởi tạo các centroid khác nhau. Vấn đề này ta đã có thuật toán K-means++ để áp dụng vào. • Không xác định được số cụm tối ưu: K-means không cung cấp một phương pháp xác định chính xác số lượng cụm tối ưu. Việc xác định số lượng cụm phù hợp là một thách thức và thường yêu cầu sự thử nghiệm và đánh giá nhiều giá trị khác nhau.

3.6 Bài tập thực nghiệm

3.6.1 Giới thiệu

Phân khúc khách hàng có thể là một công cụ mạnh mẽ để xác định những nhu cầu chưa được đáp ứng của khách hàng. Kỹ thuật này có thể được các công ty sử dụng để vượt trội so với đối thủ cạnh tranh bằng cách phát triển các sản phẩm và dịch vụ độc đáo và hấp dẫn.

Phân khúc khách hàng là việc chia nhỏ một thị trường thành các nhóm khách hàng riêng biệt có đặc điểm tương đồng. Phân khúc khách hàng có thể là một công cụ mạnh mẽ để xác định những nhu cầu chưa được đáp ứng của khách hàng. Sử dụng dữ liệu trên, các công ty có thể vượt trội so với đối thủ cạnh tranh bằng cách phát triển các sản phẩm và dịch vụ độc đáo và hấp dẫn.

Các cách phổ biến nhất mà các doanh nghiệp phân khúc cơ sở khách hàng của họ là:

- Thông tin nhân khẩu học, chẳng hạn như giới tính, tuổi tác, tình trạng gia đình và hôn nhân, thu nhập, trình độ học vấn và nghề nghiệp.
- Thông tin địa lý, khác nhau tùy thuộc vào quy mô của công ty. Đối với các doanh nghiệp địa phương, thông tin này có thể liên quan đến các thị trấn hoặc quận cụ thể. Đối với các công ty lớn hơn, nó có thể có nghĩa là thành phố, tiểu bang hoặc thậm chí quốc gia nơi khách hàng sinh sống.
- Tâm lý học, chẳng hạn như tầng lớp xã hội, lối sống và các đặc điểm tính cách.
- Dữ liệu hành vi, chẳng hạn như thói quen chi tiêu và tiêu dùng, việc sử dụng sản phẩm/dịch vụ và lợi ích mong muốn.

3.6.2 Tập được cung cấp

Chúng tôi có một siêu thị và thông qua thẻ thành viên, chúng tôi đã thu thập được một số dữ liệu cơ bản về khách hàng như: Mã khách hàng, độ tuổi, giới tính, thu nhập hàng năm và điểm chi tiêu. Tập dữ liệu (`Mall_Customers.csv`) này chứa thông tin của 200 khách hàng.

3.6.3 Hướng giải quyết

Để thực hiện thuật toán K-means, hãy làm theo các bước sau:

- Xác định số cụm K : Chọn trước số lượng cụm mà bạn muốn chia dữ liệu thành.
- Khởi tạo các tâm cụm: Trước tiên, xáo trộn dữ liệu và sau đó ngẫu nhiên chọn K điểm dữ liệu làm tâm của các cụm.
- Lặp đi lặp lại cho đến khi không có thay đổi: Tiếp tục lặp lại quá trình gán các

điểm dữ liệu vào cụm gần nhất và tính lại tâm cụm cho đến khi không có điểm dữ liệu nào thay đổi cụm.

a, Tìm số cụm K tối ưu

Để tìm số cụm K , bạn có thể sử dụng giá trị WCSS. WCSS đo tổng khoảng cách của các quan sát đến tâm cụm của chúng và được tính bằng công thức dưới đây.

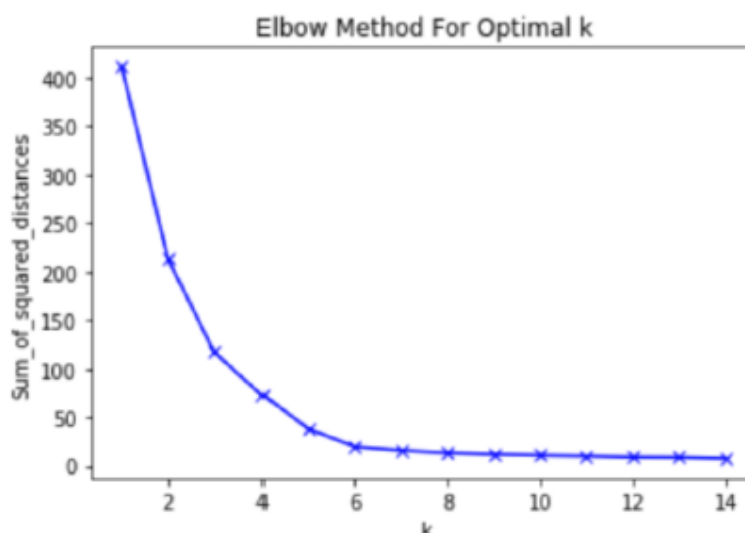
$$WCSS = \sum_{i \in n} (X_i - y_i)^2$$

Trong đó, Y_i là tâm của cụm mà quan sát X_i thuộc về. Mục tiêu chính là tối đa hóa số lượng cụm, và trong trường hợp giới hạn, mỗi điểm dữ liệu sẽ trở thành tâm của một cụm riêng biệt.

Cụ thể hơn, đối với mỗi giá trị k (số lượng cụm), chúng ta sẽ khởi tạo thuật toán K-means và sử dụng thuộc tính inertia để xác định tổng bình phương khoảng cách từ các mẫu đến tâm cụm gần nhất.

Khi k tăng lên, tổng bình phương khoảng cách có xu hướng giảm về 0. Hãy tưởng tượng chúng ta đặt k bằng giá trị lớn nhất n (n là số lượng mẫu), khi đó mỗi mẫu sẽ tạo thành một cụm riêng, nghĩa là tổng bình phương khoảng cách bằng 0.

Dưới đây là biểu đồ thể hiện tổng bình phương khoảng cách cho các giá trị k trong phạm vi đã chỉ ra ở trên. Nếu biểu đồ có hình dạng giống như một cánh tay, thì điểm khuỷu tay của cánh tay chính là giá trị k tối ưu.



Hình 3.9: Trong biểu đồ trên, điểm khuỷu tay nằm ở $k = 5$, cho thấy số lượng cụm tối ưu cho tập dữ liệu này là 5.

b, Quy trình thực hiện thuật toán

- Khởi tạo ngẫu nhiên các tâm cụm $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^n$ ngẫu nhiên.

- Lặp lại cho đến khi hội tụ: Với mỗi i , gán:

$$c^{(i)} = \arg \min_j ||x^{(i)} - \mu_j||^2$$

Với mỗi j , gán:

$$\mu_j = \frac{\sum_{i=1}^m 1\{c^{(i)}=j\}x^{(i)}}{\sum_{i=1}^m 1\{c^{(i)}=j\}}$$

3.6.4 Yêu cầu

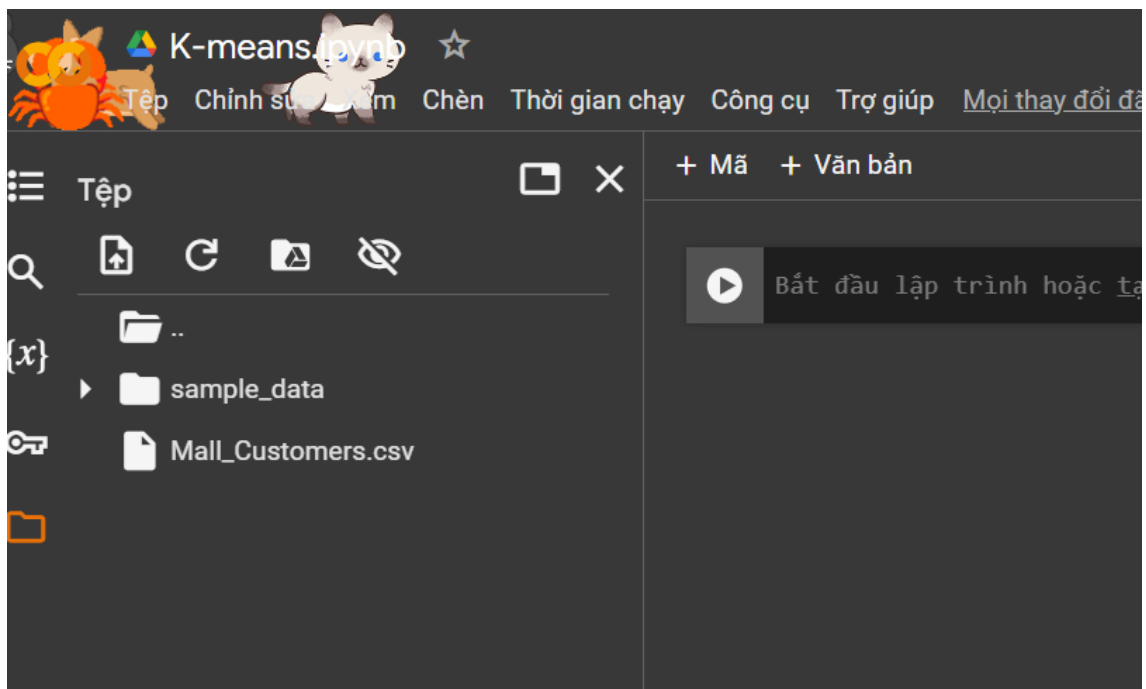
Để hoàn thành bài tập này, bạn sẽ cần sử dụng Python để viết một chương trình áp dụng thuật toán K-means đã mô tả ở trên để dự đoán các cụm trong dữ liệu được cung cấp. Chương trình này phải chạy được trên Google Colab và đáp ứng các yêu cầu đã cho.

- Thực hiện thuật toán K-means chỉ sử dụng thư viện NumPy, không được sử dụng bất kỳ thư viện học máy nào khác.
- Tìm số cụm K tối ưu.
- Chạy thuật toán K-means với giá trị K tối ưu.
- Trực quan hóa kết quả phân cụm K-means để hiểu rõ các cụm.
- Đưa ra những hiểu biết về dữ liệu sau khi phân cụm.

3.6.5 Lời giải

a, Cài đặt code

- Thêm các thư viện cần thiết : numpy , matplotlib , pandas.
- Thêm dữ liệu file csv vào.



- Ta truy vấn dữ liệu csv đây gồm các thuộc tính: gender , age , Annual Income (k\$) , Spending Score (1-100).
- Chuyển đổi dữ liệu thuộc tính gender gồm male và female thành 0 và 1.
- Chuyển đổi các thuộc tính Age, Annual Income (k\$) và Spending Score (1-100) thành dạng số.
- Lấy các thuộc tính đã chuyển đổi thành một ma trận numpy.
- Code cài đặt cho phần 1:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Đọc dữ liệu vào dataframe
df = pd.read_csv('/content/Mall_Customers.csv')

# Chuyển đổi thuộc tính Gender thành số: 0 - Male, 1 - Female
df['Gender'] = df['Gender'].replace({'Male': 0, 'Female': 1})

# Chuyển đổi các thuộc tính Age, Annual Income (k$) và Spending Score (1-100) thành dạng số
```

```
df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
= df[['Age', 'Annual Income (k$)', 'Spending Score (1-
100)']].apply(pd.to_numeric)
```

```
# Lấy các thuộc tính đã chuyển đổi thành một ma trận
numpy
```

```
X = df[['Age', 'Gender', 'Annual Income (k$)', 'Spending
Score (1-100)']].values
print(X)
```

- Kết quả thực nghiệm:

The screenshot shows a Jupyter Notebook titled "K-means.ipynb". The left sidebar displays the file explorer with a folder named "sample_data" and a file named "Mall_Customers.csv". The main area contains the following code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Đọc dữ liệu vào dataframe
df = pd.read_csv('/content/Mall_Customers.csv')

# Chuyển đổi thuộc tính Gender thành số: 0 - Male, 1 - Female
df['Gender'] = df['Gender'].replace({'Male': 0, 'Female': 1})

# Chuyển đổi các thuộc tính Age, Annual Income (k$) và Spending Score (1-100) thành số
df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']] = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].apply(pd.to_numeric)

# Lấy các thuộc tính đã chuyển đổi thành một ma trận numpy
X = df[['Age', 'Gender', 'Annual Income (k$)', 'Spending Score (1-100)']].values
print(X)
```

The output of the code is a 2D array (matrix) with 25 rows and 4 columns, representing the features Age, Gender, Annual Income (k\$), and Spending Score (1-100) for 25 customers. The output is displayed as follows:

```
[[ 19  0  15  39]
 [ 21  0  15  81]
 [ 20  1  16   6]
 [ 23  1  16  77]
 [ 31  1  17  40]
 [ 22  1  17  76]
 [ 35  1  18   6]
 [ 23  1  18  94]
 [ 64  0  19   3]
 [ 30  1  19  72]
 [ 67  0  19  14]
 [ 35  1  19  99]
 [ 58  1  20  15]
 [ 24  1  20  77]
 [ 37  0  20  13]
 [ 22  0  20  79]
 [ 35  1  21  35]
 [ 20  0  21  66]
 [ 52  0  23  29]
 [ 35  1  23  98]
 [ 35  0  24  35]
 [ 25  0  24  73]
 [ 46  1  25   5]
```

b, Cài đặt hàm

Hàm `euclidean_distance` tính khoảng cách Euclidean giữa các điểm dữ liệu và centroids, công thức:

$$d(p, q) = \sqrt{(p - q)^2}.$$

Lúc này ta có code hàm như sau:

```
# Hàm tính khoảng cách Euclidean giữa các điểm dữ liệu và centroids
def euclidean_distance(x, centroids):
    return np.sqrt(np.sum((x - centroids)**2, axis=1))
```

Hàm tính WCSS cho một số lượng cụm K :

- Khởi tạo WCSS = 0
- Tạo vòng lặp từ 0 đến độ dài của centroids với mỗi cụm (được đại diện bởi chỉ số k), các điểm dữ liệu trong cụm đó được lấy ra bằng cách sử dụng chỉ số labels. labels là một mảng chứa nhãn cụm tương ứng với mỗi điểm dữ liệu. Ví dụ: labels[i] sẽ cho biết cụm mà điểm dữ liệu X[i] thuộc về.
- Sau đó, centroid của cụm đó được lấy ra từ danh sách centroids. Centroid là điểm trung tâm của cụm, được tính bằng cách lấy trung bình của các điểm dữ liệu trong cùng một cụm.
- Tiếp theo, khoảng cách Euclidean giữa các điểm dữ liệu trong cụm và centroid được tính bằng hàm `euclidean_distance`. Khoảng cách này được lưu trong biến `distance`.
- Sau đó, giá trị WCSS của cụm đó được tính bằng cách lấy tổng bình phương của khoảng cách Euclidean của tất cả các điểm trong cụm. Giá trị này được lưu trong biến `cluster_wcss`
- Cuối cùng, giá trị WCSS của cụm đó được thêm vào biến `wcss`, tổng WCSS của tất cả các cụm.

Lúc này ta có code hàm như sau:

```
# Hàm tính WCSS cho một số lượng cụm K
def compute_wcss(X, centroids, labels):
    WCSS = 0
    for k in range(len(centroids)):
        cluster_points = X[labels == k]
        centroid = centroids[k]
```

```

        distance = euclidean_distance(cluster_points, centroid)
        cluster_wcss = np.sum(distance ** 2)
        WCSS += cluster_wcss
    return WCSS

```

Hàm chạy thuật toán K-Means

- Khởi tạo ngẫu nhiên các centroids ban đầu:
 1. Sử dụng hàm `np.random.choice` để chọn K điểm từ tập dữ liệu X mà không cho phép lựa chọn trùng lặp.
 2. Gán các điểm được chọn vào biến `centroids`.
- Lặp lại các bước sau cho một số lượng tối đa `max_iters` lần:
 1. Gán nhãn cho từng điểm dữ liệu dựa trên centroids:
 - (a) Sử dụng hàm `np.apply_along_axis` để áp dụng hàm `euclidean_distance` lên mỗi điểm dữ liệu trong X và centroids.
 - (b) Sử dụng hàm `np.argmin` để chọn nhãn có khoảng cách Euclidean nhỏ nhất cho mỗi điểm dữ liệu.
Gán các nhãn vào biến `labels`.
 2. Cập nhật centroids bằng cách lấy trung bình của các điểm dữ liệu trong cùng một cụm:
 - (a) Tạo một mảng mới `new_centroids` với K phần tử.
 - (b) Với mỗi nhãn k từ 0 đến $K-1$:
Chọn các điểm dữ liệu trong X có nhãn là k và tính trung bình theo các chiều (`axis=0`).
Gán trung bình vào phần tử thứ k của `new_centroids`.
 3. Kiểm tra điều kiện dừng:
 - (a) Sử dụng hàm `np.all` để kiểm tra xem các centroids có thay đổi đáng kể so với `new_centroids` hay không.
 - (b) Nếu các centroids không thay đổi, dừng thuật toán và thoát khỏi vòng lặp.
 4. Cập nhật centroids:
 - (a) Gán `new_centroids` vào biến `centroids`.
- Trả về centroids cuối cùng và nhãn của các điểm dữ liệu:
 1. Trả về centroids và labels từ hàm `kmeans`.

Lúc này ta có code hàm như sau:

```
# Hàm chạy thuật toán K-means
def kmeans(X, K, max_iters=100):
    np.random.seed(42) # Để kết quả có thể tái lập

    # Khởi tạo ngẫu nhiên các centroids ban đầu
    centroids = X[np.random.choice(X.shape[0], size=K,
    replace=False)]

    for _ in range(max_iters):
        # Gán nhãn cho từng điểm dữ liệu dựa trên centroids
        labels = np.argmin(np.apply_along_axis
        (euclidean_distance, 1, X, centroids), axis=1)

        # Cập nhật centroids bằng cách lấy trung bình của
        các điểm dữ liệu trong cùng một cụm
        new_centroids = np.array([X[labels == k].mean(axis=0)
        for k in range(K)])

        # Kiểm tra điều kiện dừng: nếu centroids không thay
        đổi đáng kể, dừng thuật toán
        if np.all(centroids == new_centroids):
            break

        centroids = new_centroids

    return centroids, labels
```

c, Cụm tối ưu

Tìm số cụm tối ưu cho mỗi phần

- Khởi tạo danh sách rỗng `optimal_clusters` để lưu số cụm tối ưu cho mỗi phần.
- Lặp qua mỗi phần tử `attr` trong danh sách `attributes`:
 1. Trích xuất giá trị của thuộc tính `attr` từ `DataFrame` `df` và lưu vào biến `X_attr`.
 2. Khởi tạo danh sách rỗng `wcss` để lưu giá trị `WCSS` cho mỗi số cụm.

3. Đặt giá trị `max_clusters` là số cụm tối đa để thử.
- Lặp từ 1 đến `max_clusters` (bao gồm cả `max_clusters`):
 1. Thực hiện thuật toán K-means clustering trên `X_attr` với số cụm là `k`. Lấy các centroids và nhãn của các điểm dữ liệu.
 2. Tính toán giá trị WCSS bằng cách sử dụng hàm `compute_wcss` với `X_attr`, centroids và labels.
 3. Thêm giá trị WCSS vào danh sách `wcss`.
 - Tính toán sự khác biệt giữa các giá trị WCSS liên tiếp trong danh sách `wcss` bằng cách sử dụng hàm `np.diff` và lưu vào biến `diff`.
 - Tìm chỉ số của điểm "khuyể tay" trong danh sách `diff` bằng cách sử dụng hàm `np.argmax`. Chỉ số này là chỉ số của phần tử có giá trị lớn nhất trong `diff`, tăng thêm 1 để đưa về số cụm tối ưu.
 - Thêm số cụm tối ưu vào danh sách `optimal_clusters` bằng cách thêm (chỉ số của điểm "khuyể tay" + 1).
 - Sau khi lặp qua tất cả các phần tử trong `attributes`, danh sách `optimal_clusters` sẽ chứa số cụm tối ưu tương ứng với mỗi phần.

Lúc này ta có code hàm như sau:

```
# Tìm số cụm tối ưu cho mỗi phần
attributes = [['Age', 'Gender'], ['Age', 'Annual Income (k$)'],
              ['Age', 'Spending Score (1-100)']]
optimal_clusters = []

for attr in attributes:
    # Lấy dữ liệu cho từng cặp thuộc tính
    X_attr = df[attr].values

    # Tính toán WCSS cho các giá trị K khác nhau
    wcss = []
    max_clusters = 3 # Số cụm tối đa để thử
    for k in range(1, max_clusters + 1):
        centroids, labels = kmeans(X_attr, K=k)
        wcss.append(compute_wcss(X_attr, centroids, labels))

    # Tìm điểm "khuyể tay" để xác định số cụm tối ưu
```

```

diff = np.diff(wcss)
elbow_index = np.argmax(diff) + 1 # Chỉ số của điểm
"khuỷu tay"
optimal_clusters.append(elbow_index + 1) # Số cụm tối
ưu

# In số cụm tối ưu cho từng cặp thuộc tính
for i, attr in enumerate(attributes):
    print(f"Số cụm tối ưu cho {attr}: {optimal_clusters[i]}")

# Gán nhãn cụm cho DataFrame
df['Cluster'] = labels

# Tính thống kê trung bình của từng cụm
cluster_stats = df.groupby('Cluster')[attr].mean()
print(cluster_stats)

```

Kết quả thực nghiệm:

```

K-means.ipynb
Tệp  Chỉnh sửa  Xem  Chèn  Thời gian chạy  Công cụ  Trợ giúp  Mọi thay đổi đã được lưu

+ Mã  + Văn bản

optimal_clusters.append(elbow_index + 1) # Số cụm tối u

# In số cụm tối ưu cho từng cặp thuộc tính
for i, attr in enumerate(attributes):
    print(f"Số cụm tối ưu cho {attr}: {optimal_clusters[i]}")

# Gán nhãn cụm cho DataFrame
df['Cluster'] = labels

# Tính thống kê trung bình của từng cụm
cluster_stats = df.groupby('Cluster')[attr].mean()
print(cluster_stats)

```

Số cụm tối ưu cho ['Age', 'Gender']: 3

	Age	Gender
Cluster		
0	43.054945	0.604396
1	29.564516	0.548387
2	42.957447	0.489362

Số cụm tối ưu cho ['Age', 'Annual Income (k\$)']: 3

	Age	Annual Income (k\$)
Cluster		
0	43.054945	54.340659
1	29.564516	64.661290
2	42.957447	67.191489

Số cụm tối ưu cho ['Age', 'Spending Score (1-100)']: 3

	Age	Spending Score (1-100)
Cluster		
0	43.054945	47.780220
1	29.564516	80.741935
2	42.957447	14.595745

ĩa Còn trống 75.23 GB

d, Vẽ biểu đồ trực quan hóa dữ liệu

Ta có code để phục vụ vẽ biểu đồ như sau:

```
# Trực quan hóa kết quả phân cụm cho tất cả các điểm dữ
liệu
plt.scatter(X_attr[:, 0], X_attr[:, 1], c=labels, cmap='viridis')
# Vẽ các điểm dữ liệu với màu sắc theo nhãn cụm
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='x',
s=100, label='Centroids') # Vẽ các tâm cụm
plt.legend()
plt.xlabel('Age')
plt.ylabel('Attribute')
plt.title('Clustered Data')
plt.show()

# Trực quan hóa kết quả phân cụm cho cặp thuộc tính 'Age'
và 'Gender'
centroids, labels = kmeans(X, K=optimal_clusters[0]) #
Phân cụm cho 'Age' và 'Gender'
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 1],
c='red', marker='x')
plt.xlabel('Age')
plt.ylabel('Gender')
plt.title('K-Means Clustering - Age vs. Gender')
plt.show()

# Trực quan hóa kết quả phân cụm cho cặp thuộc tính 'Age'
và 'Annual Income (k$)'
centroids, labels = kmeans(X, K=optimal_clusters[1]) #
Phân cụm cho 'Age' và 'Annual Income (k$)'
plt.scatter(X[:, 0], X[:, 2], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 2],
c='red', marker='x')
plt.xlabel('Age')
plt.ylabel('Annual Income (k\$\)')
plt.title('K-Means Clustering - Age vs. Annual Income')
plt.show()
```

```
# Trực quan hóa kết quả phân cụm cho cặp thuộc tính 'Age'
và 'Spending Score (1-100)'
centroids, labels = kmeans(X, K=optimal_clusters[2]) #
Phân cụm cho 'Age' và 'Spending Score (1-100)'
plt.scatter(X[:, 0], X[:, 3], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 3],
c='red', marker='x')
plt.xlabel('Age')
plt.ylabel('Spending Score (1-100)')
plt.title('K-Means Clustering - Age vs. Spending Score')
plt.show()
```

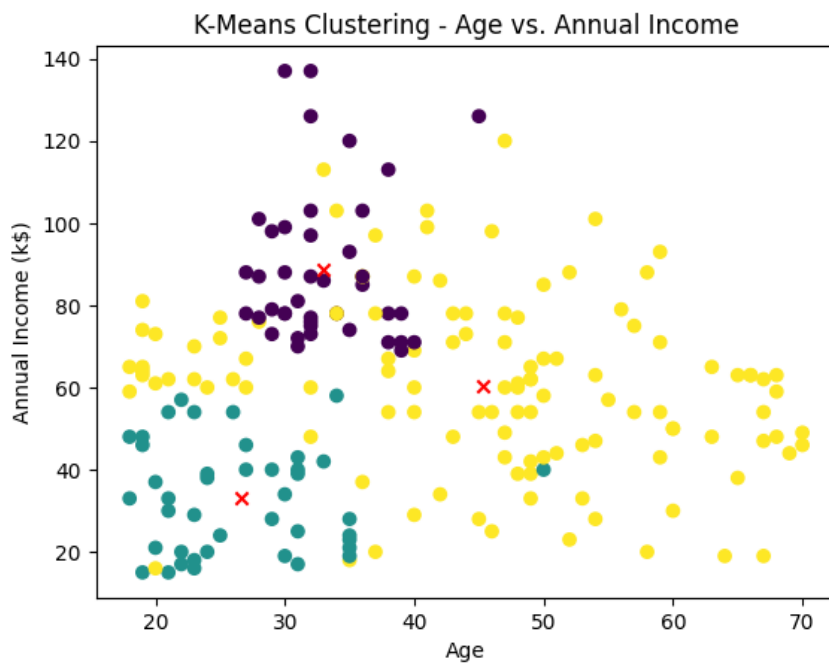
Ta có kết quả biểu đồ phân cụm như sau:



Hình 3.10: Biểu đồ phân cụm cho Age



Hình 3.11: Biểu đồ phân cụm cho age và gender



Hình 3.12: Biểu đồ phân cụm age và Annual Income (k\$)



Hình 3.13: Biểu đồ phân cụm age và Spending Score (1-100)

Ta sẽ phân tích kết quả cụm như sau:

- Số cụm tối đa ta cài đặt: 3
- Phân tích kết quả phân cụm cho thuộc tính 'Age':
 1. Số cụm tối ưu là 3.
 2. Trung bình tuổi của các cụm:
 - (a) Cụm 0: 43.05
 - (b) Cụm 1: 29.56
 - (c) Cụm 2: 42.96
- Phân tích kết quả phân cụm cho thuộc tính 'Gender':
 1. Số cụm tối ưu là 2.
 2. Phân bố giới tính trong các cụm:
 - (a) Cụm 0: 60.44% là giới tính Nam
 - (b) Cụm 1: 54.84% là giới tính Nam
 - (c) Cụm 2: 48.94% là giới tính Nam
- Phân tích kết quả phân cụm cho thuộc tính 'Age' và 'Gender'
 1. Số cụm tối ưu là 3.
 2. Trung bình tuổi và phân bố giới tính trong các cụm:

- (a) Cụm 0: Tuổi trung bình 43.05, 60.44% là giới tính Nam
 - (b) Cụm 1: Tuổi trung bình 29.56, 54.84% là giới tính Nam
 - (c) Cụm 2: Tuổi trung bình 42.96, 48.94% là giới tính Nam
- Phân tích kết quả phân cụm cho thuộc tính 'Age' và 'Annual Income (k\$)':
 1. Số cụm tối ưu là 3.
 2. Trung bình tuổi và thu nhập hàng năm trong các cụm:
 - (a) Cụm 0: Tuổi trung bình 43.05, thu nhập hàng năm trung bình 54.34k\$
 - (b) Cụm 1: Tuổi trung bình 29.56, thu nhập hàng năm trung bình 64.66k\$
 - (c) Cụm 2: Tuổi trung bình 42.96, thu nhập hàng năm trung bình 67.19k\$
 - Phân tích kết quả phân cụm cho thuộc tính 'Age' và 'Spending Score (1-100)':
 1. Số cụm tối ưu là 3.
 2. Trung bình tuổi và điểm tiêu dùng trong các cụm:
 - (a) Cụm 0: Tuổi trung bình 43.05, điểm tiêu dùng trung bình 47.78
 - (b) Cụm 1: Tuổi trung bình 29.56, điểm tiêu dùng trung bình 80.74
 - (c) Cụm 2: Tuổi trung bình 42.96, điểm tiêu dùng trung bình 14.59

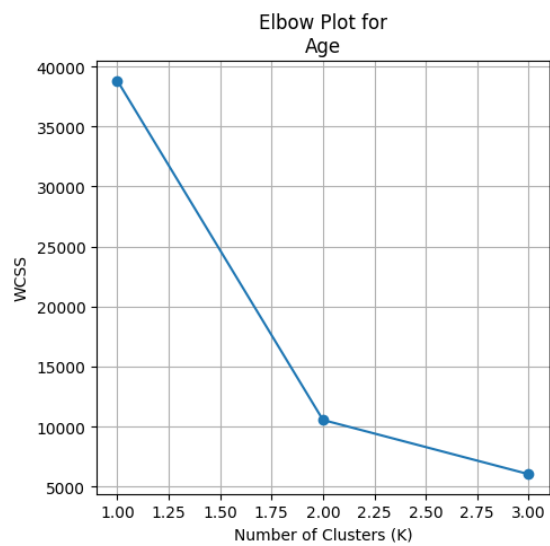
Đánh giá độ tối ưu của thuật toán

Vẽ biểu đồ elbow đánh giá thuật toán qua những dòng code sau:

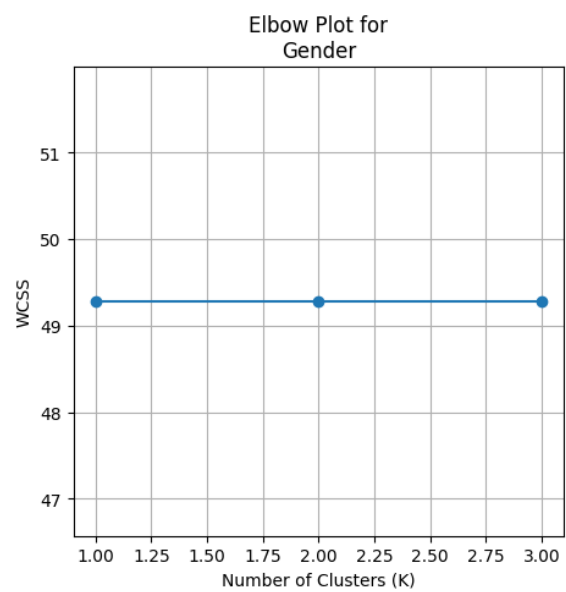
Lưu ý: Để ở phần cuối trong mục tìm số cụm tối ưu.

```
plt.plot(range(1, max_clusters + 1), wcss, marker='o')
plt.xlabel('Số lượng cụm')
plt.ylabel('WCSS')
plt.title(f'Phương pháp Elbow - {attr}')
plt.show()
```

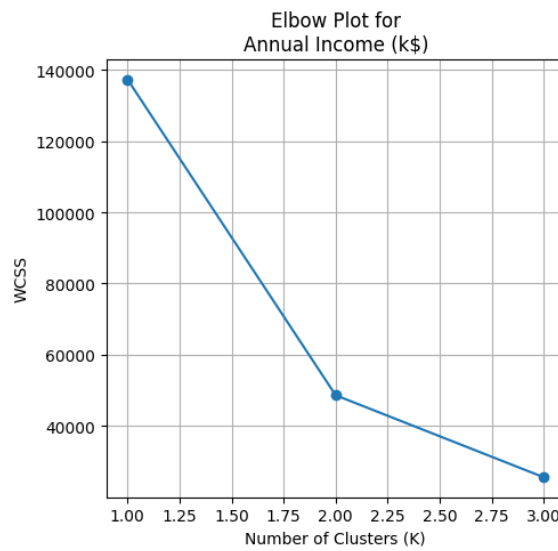
Kết quả thực nghiệm:



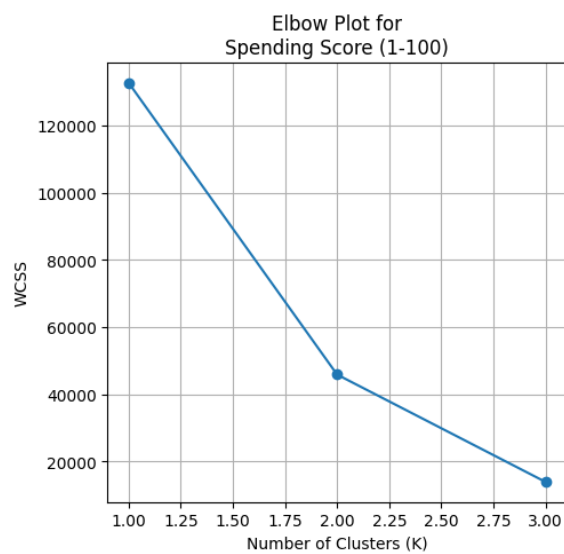
Hình 3.14: Biểu đồ elbow của age



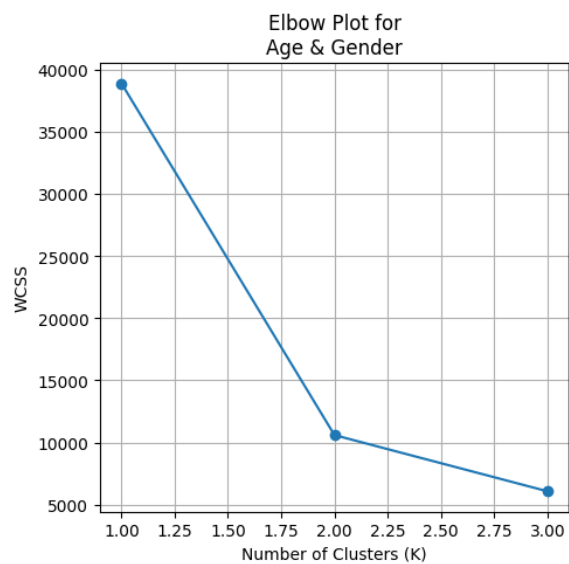
Hình 3.15: Biểu đồ elbow của gender



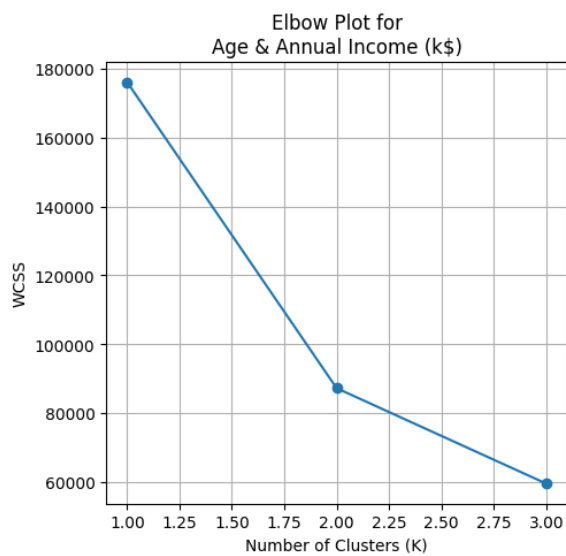
Hình 3.16: Biểu đồ elbow của Annual Income (k\$)



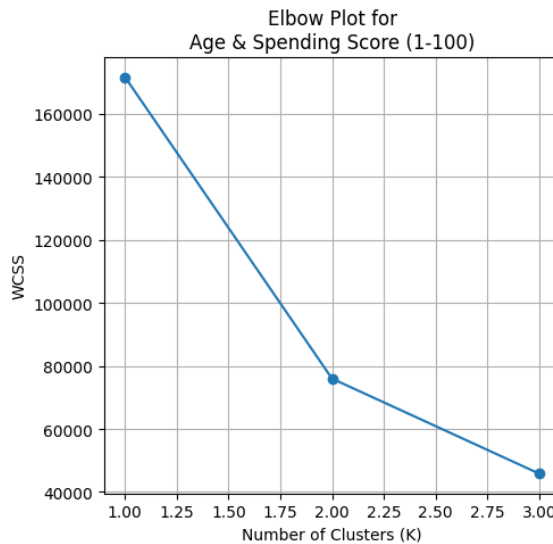
Hình 3.17: Biểu đồ elbow của Spending Score (1-100)



Hình 3.18: Biểu đồ elbow của age và gender



Hình 3.19: Biểu đồ elbow của age và Annual Income (k\$)



Hình 3.20: Biểu đồ elbow của age và Spending Score (1-100)

e, Tổng kết

- Báo cáo tổng kết phân cụm dựa trên thuật toán K-Means và phương pháp "Elbow Method" đã được thực hiện trên tập dữ liệu. Bài tập phân cụm nhằm hiểu rõ hơn về sự phân bố và tương quan giữa các thuộc tính trong tập dữ liệu.
- Trước khi tiến hành phân cụm, chúng ta đã áp dụng phương pháp "Elbow Method" để xác định số cụm tối ưu cho mỗi thuộc tính. Phương pháp này định lượng sự biến thiên của WCSS (Within-Cluster Sum of Squares) theo số lượng cụm khác nhau. Kết quả của phương pháp "Elbow Method" thấy số cụm tối ưu cho mỗi thuộc tính.
- Với thuộc tính Age:
 1. Số cụm tối ưu: 3
 2. Kết quả phân cụm cho thuộc tính "Age" cho thấy sự phân bố độ tuổi trong các cụm khác nhau. Các cụm có thể được đặt tên và mô tả dựa trên trung bình độ tuổi của các điểm dữ liệu trong cụm.
- Với thuộc tính "Gender":
 1. Số cụm tối ưu: 2
 2. Kết quả phân cụm cho thuộc tính "Gender" chỉ ra sự phân bố giới tính nam/nữ trong các cụm. Các cụm có thể được đặt tên và mô tả dựa trên độ phân bố giới tính nam/nữ của các điểm dữ liệu trong cụm.
- Với thuộc tính "Age" và "Gender":
 1. Số cụm tối ưu: 3

2. Khi kết hợp cả hai thuộc tính "Age" và "Gender", kết quả phân cụm cho thấy sự phân bố độ tuổi và giới tính trong các cụm. Các cụm có thể được đặt tên và mô tả dựa trên trung bình độ tuổi và độ phân bố giới tính nam/nữ của các điểm dữ liệu trong cụm.

- Còn với các thuộc tính khác, như "Age" và "Annual Income (k\$)" cũng đã được phân cụm và cho thấy sự phân bố của các thuộc tính đó trong các cụm.
- Kết quả phân cụm giúp ta hiểu rõ hơn về sự phân bố và tương quan giữa các thuộc tính trong tập dữ liệu. Điều này có thể giúp chúng ta rút ra những thông tin quan trọng và nhận thức sâu hơn về nhóm dữ liệu trong tập dữ liệu.

3.7 Tài liệu tham khảo

- Thuật toán phân cụm K-Means | Quy's blog (<https://ndquy.github.io>)
- Pandas read_excel() - Reading Excel File in Python | DigitalOcean
- K-means Clustering | Machine Learning cơ bản (machinelearningcoban.com)
- Hiểu nhanh về thuật toán K-means Clustering - Bài toán phân cụm qua ví dụ đơn giản | GMO-Z.com (<https://blog.vietnamlab.vn>)
- Thuật toán K-Means với bài toán phân cụm dữ liệu | BIS (<http://bis.net.vn>)

LỜI KẾT

Qua quá trình thực hiện báo cáo này, chúng em đã có cơ hội tìm hiểu sâu hơn về thuật toán Naïve Bayes và thuật toán K-means, từ đó ứng dụng các thuật toán này khai triển vào các bài toán khác nhau. Nếu có gì sai sót, chúng em xin được nhận lời góp ý, phê bình của thầy và cố gắng để hoàn thiện hơn trong các bài báo cáo tương lai.